

CS1139 Programming – I

Lecture Notes: Unit I and Unit II (Lectures 1–18)

Part A — Basics of Computers and Programming

1. Introduction to Computers

1.1 What is a computer?

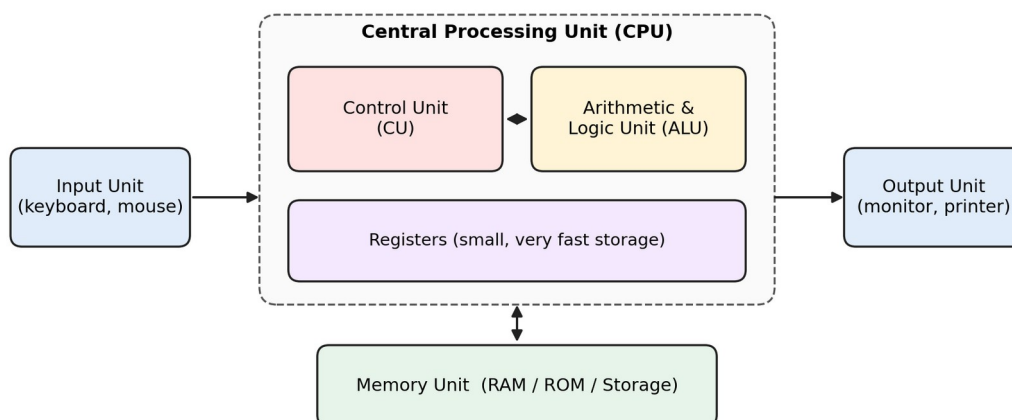
A **computer** is an electronic device that accepts data as **input**, **processes** it according to a set of instructions (a **program**), **stores** data and results, and produces meaningful **output**. The complete cycle is often summarised as **Input → Process → Output (IPO)**, with **Storage** supporting every stage.

Data is a collection of raw facts (for example, 72, 85, 91). **Information** is processed data that has meaning (for example, "the class average is 82.7").

Key characteristics of a computer:

- **Speed:** performs billions of operations per second.
- **Accuracy:** results are correct if the program and data are correct (the principle of *Garbage In, Garbage Out*, GIGO).
- **Diligence:** does not get tired or lose concentration while repeating a task millions of times.
- **Storage:** can store and recall very large amounts of data.
- **Versatility:** the same machine can do word processing, games, calculations and communication simply by running different programs.
- **No intelligence of its own:** a computer does only what it is instructed to do. Deciding *what* to instruct is the programmer's job.

1.2 Functional units (block diagram)



- **Input unit:** accepts data and instructions from the user (keyboard, mouse, scanner, microphone) and converts them into a form the computer understands (binary).
- **Central Processing Unit (CPU):** the "brain" of the computer. It contains:

- **Control Unit (CU):** fetches instructions from memory, decodes them and directs all other units. It does not process data itself.
- **Arithmetic and Logic Unit (ALU):** performs arithmetic (+, −, ×, ÷) and logical operations (comparisons such as <, >, =, and AND/OR/NOT).
- **Registers:** very small, very fast storage locations inside the CPU used to hold the data currently being worked on.
- **Memory unit:** stores data and programs. **Primary memory** (RAM – volatile, ROM – non-volatile) is directly used by the CPU; **secondary memory** (hard disk, SSD, pen drive) stores data permanently.
- **Output unit:** presents results to the user (monitor, printer, speaker).

1.3 Units of memory

A **bit** (binary digit) is the smallest unit and holds either 0 or 1. Larger units are built from it:

Unit	Equal to
1 Nibble	4 bits
1 Byte	8 bits
1 Kilobyte (KB)	1024 Bytes (2^{10} Bytes)
1 Megabyte (MB)	1024 KB
1 Gigabyte (GB)	1024 MB
1 Terabyte (TB)	1024 GB

1.4 Programming languages and translators

A **program** is a finite sequence of instructions written to perform a specific task. A **programming language** is a formal language used to write such instructions.

Level	Description	Example
Machine language (low level)	Instructions written directly in binary (0s and 1s). Executed directly by the CPU; very hard for humans to read.	10110000 01100001
Assembly language (low level)	Uses short mnemonics instead of binary codes. Needs an assembler to convert it to machine code.	MOV AL, 61h
High-level language	Close to English and mathematics; independent of the machine. Needs a compiler or interpreter .	Python, C, Java

Since the CPU understands only machine language, every program written in any other language must be **translated**:

Compiler	Interpreter
Translates the whole program at once into machine code before running it.	Translates and executes the program one statement at a time .
Errors are reported together after compilation.	Execution stops at the first error encountered.
Produces a separate executable file; execution is fast.	No separate executable; generally slower execution.
Examples: C, C++	Examples: Python, JavaScript

Python is usually described as an **interpreted** language. Internally, the Python interpreter first compiles the source code into an intermediate form called **bytecode** (.pyc), which is then executed by the **Python Virtual Machine (PVM)**.

2. Number Systems

2.1 Basic idea

A **number system** is a way of representing numbers using a fixed set of symbols (digits). The number of distinct digits used is called the **base** (or **radix**) of the system. In a **positional number system**, the value of a digit depends on its position: each position carries a weight equal to a power of the base.

Number system	Base	Digits used	Example	Python prefix
Binary	2	0, 1	$(1011)_2$	<code>0b1011</code>
Octal	8	0 – 7	$(357)_8$	<code>0o357</code>
Decimal	10	0 – 9	$(429)_{10}$	<code>429</code>
Hexadecimal	16	0 – 9, A(10), B(11), C(12), D(13), E(14), F(15)	$(2F9)_{16}$	<code>0x2F9</code>

For any number in base **b**, the weight of the digit at position **i** (counting from 0 at the right of the point) is **bⁱ**; positions to the right of the point have weights **b⁻¹**, **b⁻²**, ...

Example: $(429)_{10} = 4 \times 10^2 + 2 \times 10^1 + 9 \times 10^0 = 400 + 20 + 9$.

Computers use binary because electronic circuits have two stable states (ON/OFF, high/low voltage). Octal and hexadecimal are used as **short-hand for binary**, because $8 = 2^3$ and $16 = 2^4$: one octal digit stands for exactly 3 bits and one hex digit for exactly 4 bits.

Quick reference table (0 to 15)

Decimal	Binary (4-bit)	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E

Decimal	Binary (4-bit)	Octal	Hexadecimal
15	1111	17	F

2.2 Any base → Decimal (method of positional weights)

Method: multiply each digit by its positional weight ($\text{base}^{\text{position}}$) and add all the products.

Example 1: $(110101)_2$ to decimal

```

Position : 5   4   3   2   1   0
Digit    : 1   1   0   1   0   1
Weight   : 32  16   8   4   2   1
Value    = 32 + 16 + 0 + 4 + 0 + 1 = 53

```

Answer: $(110101)_2 = (53)_{10}$

Example 2: $(101.11)_2$ to decimal (with a fractional part)

```

=  $1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2}$ 
=  $4 + 0 + 1 + 0.5 + 0.25$ 
= 5.75

```

Example 3: $(352)_8$ to decimal

```

=  $3 \times 8^2 + 5 \times 8^1 + 2 \times 8^0$ 
=  $3 \times 64 + 5 \times 8 + 2 \times 1$ 
=  $192 + 40 + 2 = 234$ 

```

Example 4: $(2F9)_{16}$ to decimal

```

=  $2 \times 16^2 + F \times 16^1 + 9 \times 16^0$ 
=  $2 \times 256 + 15 \times 16 + 9 \times 1$ 
=  $512 + 240 + 9 = 761$ 

```

Example 5: $(1A.8)_{16}$ to decimal

```

=  $1 \times 16^1 + A \times 16^0 + 8 \times 16^{-1}$ 
=  $16 + 10 + 8/16$ 
= 26.5

```

2.3 Decimal → Any base

Integer part — repeated division: divide the number by the target base repeatedly, noting the remainder each time, until the quotient becomes 0. Read the remainders **from bottom to top** (last remainder is the most significant digit).

Fractional part — repeated multiplication: multiply the fraction by the target base; the integer part of the product is the next digit. Continue with the new fractional part until it becomes 0 (or until enough digits are obtained). Read the digits **from top to bottom**.

Example 6: $(45)_{10}$ to binary

```
2 | 45    remainder
2 | 22 -> 1    (LSB)
2 | 11 -> 0
2 | 5  -> 1
2 | 2  -> 1
2 | 1  -> 0
  | 0  -> 1    (MSB)
Reading bottom to top: 101101
```

Answer: $(45)_{10} = (101101)_2$

Example 7: $(0.625)_{10}$ to binary

```
0.625 × 2 = 1.25    -> 1
0.25  × 2 = 0.50    -> 0
0.50  × 2 = 1.00    -> 1    (fraction is now 0, stop)
Reading top to bottom: .101
```

Answer: $(0.625)_{10} = (0.101)_2$. Combining both methods, $(13.375)_{10} = (1101.011)_2$.

Example 8: $(159)_{10}$ to octal

```
8 | 159    remainder
8 | 19  -> 7
8 | 2   -> 3
  | 0   -> 2
Reading bottom to top: 237
```

Answer: $(159)_{10} = (237)_8$. Check: $2 \times 64 + 3 \times 8 + 7 = 159$ ✓

Example 9: $(2748)_{10}$ to hexadecimal

```
16 | 2748    remainder
16 | 171  -> 12 (C)
16 | 10   -> 11 (B)
  | 0     -> 10 (A)
Reading bottom to top: ABC
```

Answer: $(2748)_{10} = (ABC)_{16}$

Some fractions never terminate in another base. For example, $(0.1)_{10} = (0.000110011...)_{2}$ repeats forever. This is why Python prints $0.1 + 0.2$ as 0.30000000000000004 — the computer stores only an approximation of 0.1 in binary.

2.4 Binary ↔ Octal (groups of 3 bits)

Binary → Octal: starting from the binary point, form groups of **3 bits** (to the left for the integer part, to the right for the fraction). Pad with 0s at the ends if needed, then replace each group by its octal digit.

Octal → Binary: replace each octal digit by its 3-bit binary equivalent.

Example 10: $(11010111)_2$ to octal

```
Pad on the left : 011 010 111
Octal digits    : 3   2   7
```

Answer: $(327)_8$

Example 11: $(725)_8$ to binary

```
7 -> 111    2 -> 010    5 -> 101
```

Answer: $(111010101)_2$

2.5 Binary ↔ Hexadecimal (groups of 4 bits)

Binary → Hex: form groups of **4 bits** from the binary point outward, pad with 0s, and replace each group by its hex digit. **Hex → Binary:** replace each hex digit with its 4-bit equivalent.

Example 12: $(1011011101)_2$ to hexadecimal

```
Pad on the left : 0010 1101 1101
Hex digits      :   2   D   D
```

Answer: $(2DD)_{16}$

Example 13: $(3AF)_{16}$ to binary

```
3 -> 0011    A -> 1010    F -> 1111
```

Answer: $(001110101111)_2 = (1110101111)_2$ (leading zeros can be dropped).

Example 14: $(1101.1011)_2$ to hexadecimal and octal (with fraction)

```
Hex   : 1101 . 1011      -> D . B      = (D.B)16
Octal : 001 101 . 101 100 -> 1 5 . 5 4 = (15.54)8
      (pad the integer part on the LEFT, the fraction on the RIGHT)
```

2.6 Octal ↔ Hexadecimal (through binary)

There is no direct grouping between base 8 and base 16, so use binary as a bridge: **Octal → Binary (3 bits each) → regroup into 4 bits → Hex**, and the reverse for Hex → Octal. (Going through decimal also works but takes longer.)

Example 15: $(574)_8$ to hexadecimal

```
Octal to binary : 5 -> 101    7 -> 111    4 -> 100    => 101111100
Regroup in 4s   : 0001 0111 1100
Hex digits      :   1   7   C
```

Answer: $(17C)_{16}$. Check: $(574)_8 = 380$ and $(17C)_{16} = 256 + 112 + 12 = 380$ ✓

Example 16: $(B6)_{16}$ to octal

```

Hex to binary   : B -> 1011   6 -> 0110   => 10110110
Regroup in 3s   : 010 110 110
Octal digits    : 2   6   6

```

Answer: $(266)_8$ **Summary of all conversions**

From \ To	Binary	Octal	Decimal	Hexadecimal
Binary	—	Group 3 bits	Positional weights	Group 4 bits
Octal	Each digit → 3 bits	—	Positional weights	Via binary
Decimal	Divide by 2 / multiply by 2	Divide by 8 / multiply by 8	—	Divide by 16 / multiply by 16
Hexadecimal	Each digit → 4 bits	Via binary	Positional weights	—

2.7 Number systems in Python

Python lets you write integers directly in binary, octal or hex and provides built-in functions to convert between them. These are useful for checking your hand calculations.

```

print(0b101101, 0o237, 0xABC)           # literals in other bases are stored as int
print(bin(45), oct(159), hex(2748))      # decimal -> string in another base
print(int("110101", 2))                  # string in base 2 -> decimal
print(int("352", 8), int("2F9", 16))
print(format(45, "b"), format(45, "08b"), format(2748, "X")) # without prefix

```

Output

```

45 159 2748
0b101101 0o237 0xabc
53
234 761
101101 00101101 ABC

```

2.8 Practice questions — number systems

- Convert $(156)_{10}$ to binary.
- Convert $(237)_{10}$ to octal.
- Convert $(3054)_{10}$ to hexadecimal.
- Convert $(0.8125)_{10}$ to binary.
- Convert $(101110.101)_2$ to decimal.
- Convert $(1777)_8$ to decimal.
- Convert $(7E3)_{16}$ to decimal.
- Convert $(110111011)_2$ to octal and to hexadecimal.
- Convert $(4D2)_{16}$ to octal.
- Convert $(615)_8$ to hexadecimal.
- Convert $(25.3)_8$ to decimal.
- Convert $(A9.C)_{16}$ to binary and to decimal.

Answers: 1) 10011100 2) 355 3) BEE 4) 0.1101 5) 46.625 6) 1023 7) 2019 8) 673 and 1BB 9) 2322 10) 18D 11) 21.375 12) 10101001.11 and 169.75

3. Problem Solving and Computational Thinking

3.1 Steps in solving a problem with a computer

1. **Understand the problem:** identify what is given (input), what is required (output) and any conditions (constraints).
2. **Design a solution:** work out the step-by-step method — an **algorithm** — and represent it as a **flowchart** or **pseudocode**.
3. **Write the program (coding):** translate the algorithm into a programming language such as Python.
4. **Test and debug:** run the program with different inputs (including unusual ones) and remove errors.
5. **Document and maintain:** add comments and explanations; update the program when requirements change.

3.2 Computational thinking

Computational thinking is a problem-solving approach in which a problem is expressed in such a way that its solution can be carried out by a computer (or a person following precise steps). It has four pillars:

Pillar	Meaning	Example: preparing a class result sheet
Decomposition	Breaking a large problem into smaller, manageable sub-problems.	Read marks → compute totals → compute percentage → assign grades → print report.
Pattern recognition	Spotting similarities or repeated steps.	The same "compute total and percentage" steps repeat for every student, so they can be put in a loop.
Abstraction	Focusing on the important details and ignoring irrelevant ones.	Only roll number and marks matter; the student's hostel or hobbies are ignored.
Algorithm design	Writing a precise, ordered set of steps to solve the problem.	Step 1: Read marks ... Step n: Print the report.

4. Algorithms

4.1 Definition

An **algorithm** is a finite, ordered sequence of well-defined, unambiguous steps that takes some input and produces the desired output in a finite amount of time. The word comes from the name of the 9th-century Persian mathematician **al-Khwarizmi**.

An algorithm is independent of any programming language; it can be written in plain English (numbered steps), drawn as a flowchart, or written as pseudocode.

4.2 Characteristics of a good algorithm

- **Input:** zero or more well-defined inputs.
- **Output:** at least one output.
- **Definiteness:** every step is clear and has exactly one meaning.
- **Finiteness:** it must stop after a finite number of steps.

- **Effectiveness:** every step is basic enough to be carried out exactly, in finite time, with pen and paper.
- **Generality:** it should solve all problems of the same type, not just one particular set of values.

4.3 Writing algorithms — conventions

Steps are numbered and start with **Step 1: Start** and end with **Step n: Stop**. Use verbs such as **Read/Input**, **Print/Display**, **Set/Compute**, **If ... then ... else**, and **Go to Step k** (for repetition).

Algorithm Example 1: Compute simple interest.

Algorithm

```
Step 1: Start
Step 2: Read principal P, rate R, time T
Step 3: Compute  $SI = (P \times R \times T) / 100$ 
Step 4: Print SI
Step 5: Stop
```

Algorithm Example 2: Check whether a number is even or odd.

Algorithm

```
Step 1: Start
Step 2: Read N
Step 3: If  $N \bmod 2 = 0$  then Print "Even"
        else Print "Odd"
Step 4: Stop
```

Algorithm Example 3: Find the GCD (HCF) of two positive integers using Euclid's method.

Algorithm

```
Step 1: Start
Step 2: Read A, B
Step 3: If  $B = 0$  then go to Step 7
Step 4: Set  $R = A \bmod B$ 
Step 5: Set  $A = B, B = R$ 
Step 6: Go to Step 3
Step 7: Print A as the GCD
Step 8: Stop
```

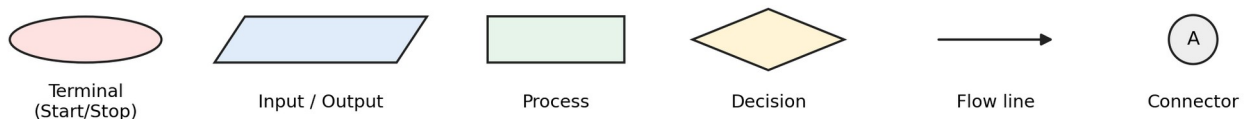
Trace for $A = 48, B = 18$: $(48, 18) \rightarrow (18, 12) \rightarrow (12, 6) \rightarrow (6, 0)$, so $GCD = 6$. Notice how "Go to Step 3" creates **repetition** — later this becomes a while loop.

5. Flowcharts

5.1 Definition

A **flowchart** is a **pictorial (graphical) representation of an algorithm** in which standard symbols represent different kinds of steps and arrows (flow lines) show the order in which the steps are carried out.

5.2 Standard symbols



Symbol	Name	Use
Oval / ellipse	Terminal	Marks Start and Stop of the flowchart.
Parallelogram	Input / Output	Reading input or printing output.
Rectangle	Process	Calculations and assignments, e.g. $\text{sum} = a + b$.
Diamond	Decision	A condition with two exits: Yes/No (True/False).
Arrow	Flow line	Shows the direction of flow.
Small circle	Connector	Joins parts of a flowchart that are far apart or continue on another page.

5.3 Rules for drawing flowcharts

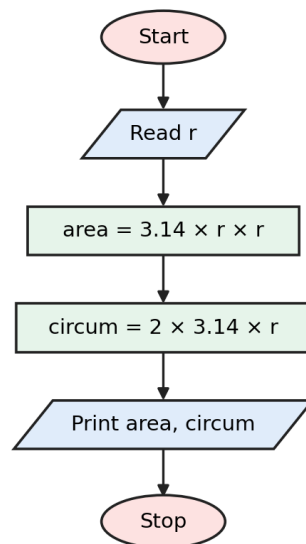
- Every flowchart has exactly one **Start** and at least one **Stop**.
- The flow is generally **top to bottom** and **left to right**.
- A process or I/O symbol has **one entry and one exit**; a decision symbol has **one entry and two exits** (Yes/No), each clearly labelled.
- Flow lines should not cross; use connectors if needed.
- The text inside a symbol should be short and clear.
- **Loops** are drawn by sending a flow line back to a decision box that is checked again.

Advantages: easy to understand and communicate, helps in finding logical errors, serves as documentation.

Limitations: large programs give very big flowcharts, and modifying them is time-consuming.

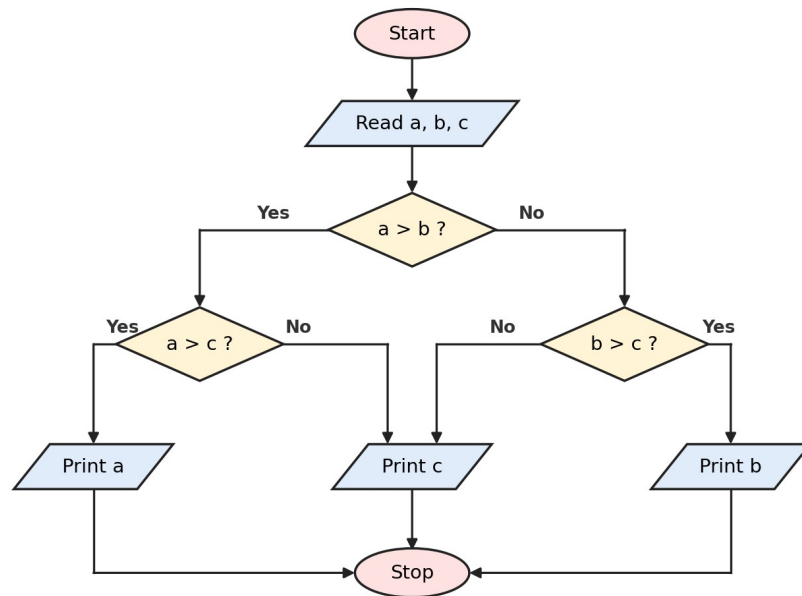
5.4 Solved flowchart examples

Flowchart 1 (sequence): Find the area and circumference of a circle of radius r .



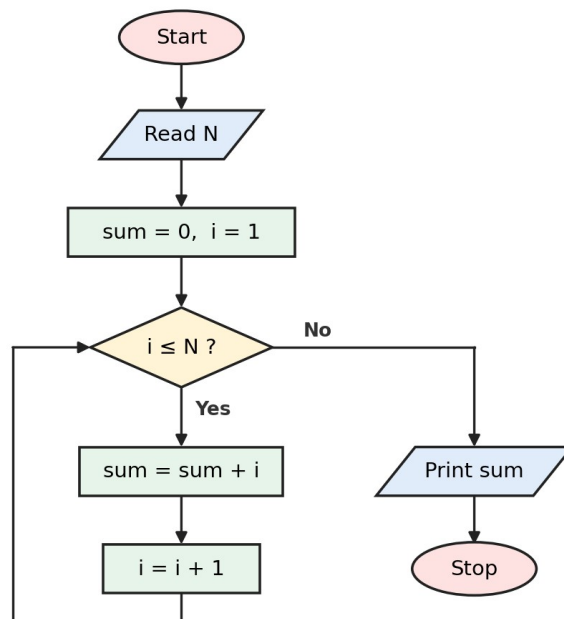
This is a purely **sequential** flowchart: every step is executed exactly once, in order.

Flowchart 2 (decision / nested decision): Find the largest of three numbers a, b and c.



If $a > b$, the answer is either a or c, so only a and c are compared next. Otherwise the answer is either b or c. A decision placed inside the branch of another decision is called a **nested decision**.

Flowchart 3 (loop): Find the sum of the first N natural numbers ($1 + 2 + \dots + N$).

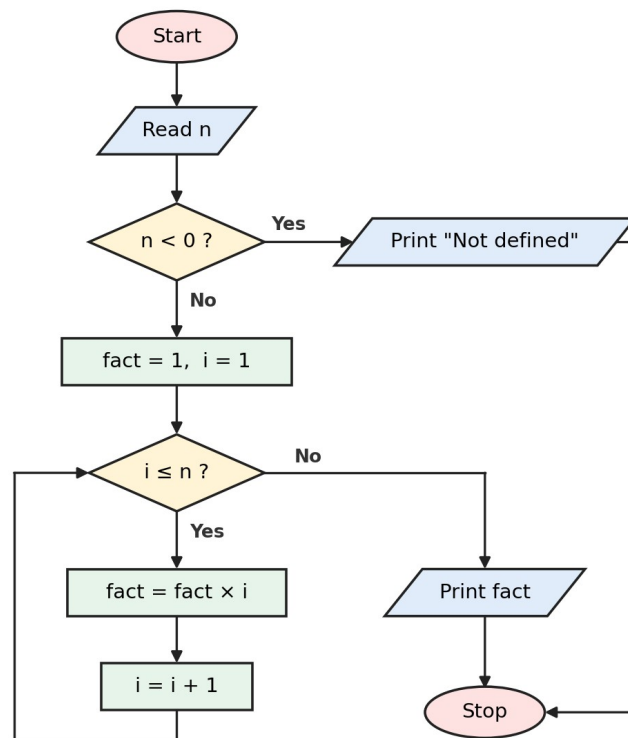


The decision $i \leq N$? is checked before every repetition. The body (add i, then increase i) repeats until the condition becomes false. The line returning to the decision is the **loop**.

Dry run (trace table) for $N = 4$:

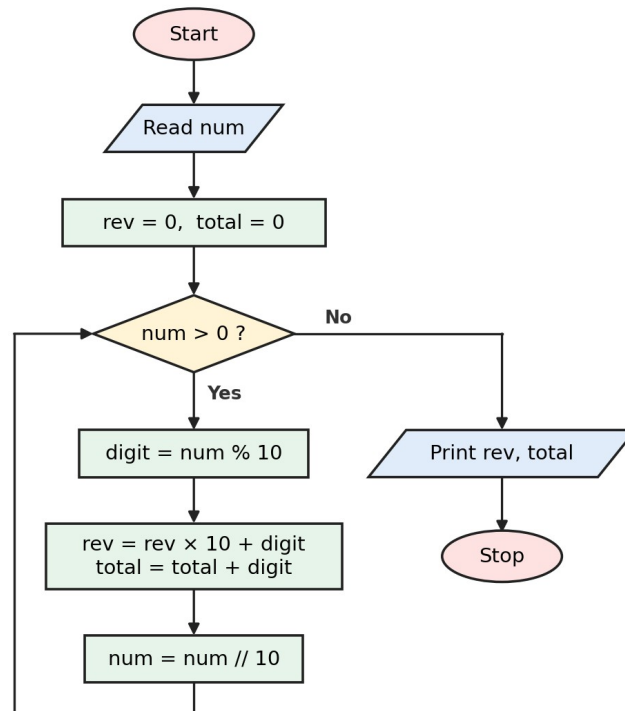
i	$i \leq N$?	sum after the step
1	Yes	$0 + 1 = 1$
2	Yes	$1 + 2 = 3$
3	Yes	$3 + 3 = 6$
4	Yes	$6 + 4 = 10$
5	No $\rightarrow$ exit	Print 10

Flowchart 4 (decision + loop): Find the factorial of n ($n! = 1 \times 2 \times \dots \times n$).



The first decision handles invalid input (factorial is not defined for negative numbers). Note that for $n = 0$ the loop body never runs and fact remains 1, which is correct since $0! = 1$.

Flowchart 5 (loop on digits): Find the reverse of a number and the sum of its digits.

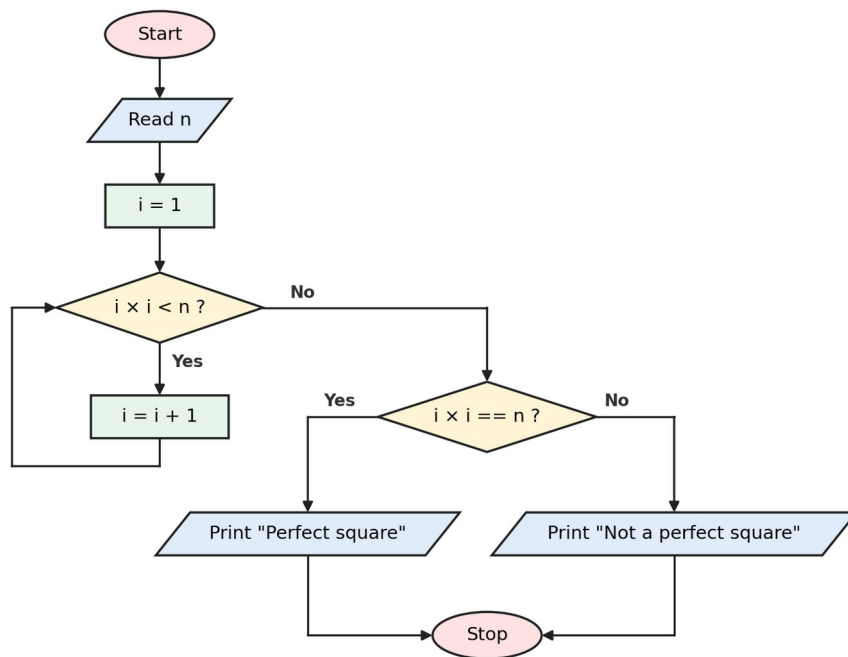


The operator % gives the remainder (last digit) and // gives the integer quotient (removes the last digit).

Dry run for num = 472:

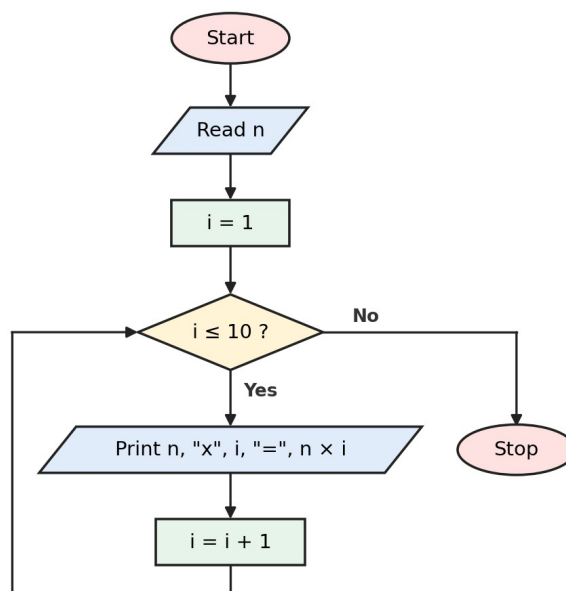
num	digit	rev	total
472	2	$0 \times 10 + 2 = 2$	2
47	7	$2 \times 10 + 7 = 27$	9
4	4	$27 \times 10 + 4 = 274$	13
0	—	loop ends	Print 274, 13

Flowchart 6 (loop followed by a decision): Check whether a positive integer n is a perfect square (e.g. $49 = 7 \times 7$), without using a square root.



The loop keeps increasing i while $i \times i$ is still smaller than n . When the loop stops, $i \times i$ is either exactly n (a perfect square) or has just gone past n (not a perfect square). For $n = 50$ the loop stops at $i = 8$ because $8 \times 8 = 64 \geq 50$, and $64 \neq 50$, so 50 is not a perfect square.

Flowchart 7 (counter-controlled loop with output inside the loop): Print the multiplication table of n .



Here the output symbol is **inside** the loop, so it is executed 10 times, producing lines such as $7 \times 1 = 7$, $7 \times 2 = 14$, ..., $7 \times 10 = 70$.

5.5 Practice questions — flowcharts

Draw a flowchart for each of the following:

1. Convert a temperature from Celsius to Fahrenheit ($F = 9C/5 + 32$).

2. Swap two numbers (a) using a third variable (b) without a third variable.
3. Find the smallest of three numbers.
4. Check whether a year is a leap year.
5. Read the price and quantity of an item; if the bill exceeds ₹1000 give a 10% discount, otherwise no discount. Print the final bill.
6. Print all even numbers from 2 to 50.
7. Find the sum of the squares of the first N natural numbers.
8. Count the number of digits in a given positive integer.
9. Print the first N terms of the Fibonacci series (0, 1, 1, 2, 3, 5, ...).
10. Read numbers one by one until the user enters 0, then print how many positive and how many negative numbers were entered.
11. Find x raised to the power n using repeated multiplication.
12. Check whether a number is a palindrome (reads the same forwards and backwards, e.g. 1221).

6. Pseudocode

6.1 Definition

Pseudocode ("false code") is an informal, English-like description of an algorithm that uses the **structure of a programming language** (IF, WHILE, FOR ...) but ignores its exact syntax. It is meant to be read by humans, not executed by a computer.

6.2 Conventions

- Write one statement per line; use **indentation** to show which statements belong to a block.
- Keywords are written in capitals: **READ / INPUT, PRINT / DISPLAY, SET, IF ... THEN ... ELSE ... ENDIF, WHILE ... DO ... ENDWHILE, FOR ... TO ... DO ... ENDFOR, REPEAT ... UNTIL.**
- Use meaningful variable names and $\leftarrow$ or = for assignment.
- Keep it language-independent: do not use Python-specific functions.

Pseudocode Example 1: Print the age group of a person.

Pseudocode

```
BEGIN
  READ age
  IF age < 0 THEN
    PRINT "Invalid age"
  ELSE IF age < 13 THEN
    PRINT "Child"
  ELSE IF age < 20 THEN
    PRINT "Teenager"
  ELSE IF age < 60 THEN
    PRINT "Adult"
  ELSE
    PRINT "Senior citizen"
  ENDIF
END
```

Pseudocode Example 2: Print the first N terms of the Fibonacci series.

Pseudocode

```
BEGIN
  READ N
  SET a ← 0, b ← 1, count ← 1
  WHILE count ≤ N DO
    PRINT a
    SET next ← a + b
    SET a ← b
    SET b ← next
    SET count ← count + 1
  ENDWHILE
END
```

Pseudocode Example 3: Read N numbers and find the largest.

Pseudocode

```
BEGIN
  READ N
  READ x
  SET largest ← x
  FOR i ← 2 TO N DO
    READ x
    IF x > largest THEN
      SET largest ← x
    ENDIF
  ENDFOR
  PRINT largest
END
```

Pseudocode Example 4: Count how many numbers between 1 and 100 are divisible by 3 or 5.

Pseudocode

```
BEGIN
  SET count ← 0
  FOR i ← 1 TO 100 DO
    IF (i MOD 3 = 0) OR (i MOD 5 = 0) THEN
      SET count ← count + 1
    ENDIF
  ENDFOR
  PRINT count
END
```

(The answer printed is 47.)

6.3 Algorithm vs flowchart vs pseudocode

Algorithm (steps)	Flowchart	Pseudocode
Numbered steps in plain English.	Diagram using standard symbols.	Structured English using programming constructs.
Easy to write; can become ambiguous for complex logic.	Easiest to understand visually; tedious to draw and modify for large problems.	Compact and closest to real code; easy to convert into a program.
Repetition shown with "Go to Step k".	Repetition shown with a back-arrow to a decision.	Repetition shown with WHILE / FOR blocks.

6.4 Practice questions — algorithms and pseudocode

Write both an algorithm (numbered steps) and pseudocode for:

1. Calculating the total and percentage of marks in five subjects.
2. Checking whether a triangle is valid given its three angles.
3. Finding the roots of a quadratic equation $ax^2 + bx + c = 0$.
4. Printing the sum of the series $1 + 1/2 + 1/3 + \dots + 1/N$.
5. Finding the LCM of two numbers.
6. Finding the second largest of N numbers entered one by one.
7. Counting how many of N entered numbers are even.
8. Converting a decimal number to binary using repeated division.
9. Calculating an electricity bill: first 100 units at ₹5/unit, next 100 at ₹7/unit, remaining at ₹10/unit.
10. Printing all numbers between 1 and N that are divisible by 7 but not by 5.

Part B — Fundamentals of Python

7. Beginning with Python

7.1 What is Python?

Python is a high-level, general-purpose, interpreted programming language created by **Guido van Rossum** and first released in **1991**. It is named after the comedy group *Monty Python*, not the snake. The current major version is **Python 3**; Python 2 is no longer supported.

Features of Python:

- **Simple and readable:** syntax is close to English; blocks are shown by indentation instead of braces.
- **Interpreted:** no separate compilation step; code runs line by line, which makes testing easy.
- **Dynamically typed:** the type of a variable is decided at run time; no declarations are needed.
- **Free and open source**, and **portable** (the same program runs on Windows, Linux and macOS).
- **Large standard library** ("batteries included") and thousands of third-party packages (NumPy, Pandas, Matplotlib ...).
- **Multi-paradigm:** supports procedural, object-oriented and functional styles.

Applications: web development, data science, machine learning and AI, automation and scripting, scientific computing, cybersecurity tools, and education.

7.2 How a Python program runs

```
hello.py (source code)
  |   compiled by the Python interpreter
  v
hello.pyc (bytecode)
  |   executed by the Python Virtual Machine (PVM)
  v
Output
```

7.3 Two ways of working

Interactive mode	Script mode
Statements are typed at the >>> prompt (the Python shell or the Spyder IPython console) and executed immediately.	The whole program is written in a file with the .py extension and executed together (Run / F5 in Spyder).
Good for testing small pieces of code and quick calculations.	Used for real programs; the code can be saved, edited and reused.
The value of an expression is displayed automatically.	Values are displayed only when print() is used.

In this course we use the **Spyder IDE** (Integrated Development Environment), which provides an editor for scripts, an IPython console for interactive work, and a variable explorer.

7.4 The first program

```
print("Hello, World!")
print("Welcome to CS1139")
```

Output

```
Hello, World!  
Welcome to CS1139
```

print() is a built-in **function** that displays its arguments on the screen. Text enclosed in quotes is called a **string**.

8. Parts of a Python Program

A Python program is a sequence of **statements**; statements contain **expressions**, and everything is built from **tokens**. **Comments** explain the code and **indentation (whitespace)** groups statements into blocks. The small program below shows these parts:

```
# Program to find the area of a circle      <- comment  
PI = 3.14159                               # <- assignment statement  
  
radius = float(input("Enter radius: "))    # <- input statement  
area = PI * radius ** 2                    # <- expression on the right side  
print("Area =", round(area, 2))            # <- function call statement
```

Output

```
Enter radius: 5  
Area = 78.54
```

8.1 Statements

A **statement** is a complete instruction that the Python interpreter can execute. It performs an action but does not necessarily produce a value.

- **Simple statements** fit on one line: assignment (`x = 5`), `print(x)`, `pass`, `break`, `continue`, `del x`.
- **Compound statements** contain other statements in an indented block: `if`, `for`, `while`, `def`. They begin with a header line ending in a colon (`:`).

Writing statements:

```
a = 10; b = 20; print(a + b)                # several statements on one line using ;  
total = 1 + 2 + 3 + \                       # explicit continuation with \  
        4 + 5  
marks = (90 + 85 +                           # implicit continuation inside brackets  
        78)  
print(total, marks)
```

Output

```
30  
15 253
```

8.2 Expressions

An **expression** is a combination of values (literals), variables, operators and function calls that the interpreter **evaluates to produce a single value**. Every expression has a value and a type.

```
x = 7
print(x + 3)           # arithmetic expression -> 10
print(x > 5)           # relational expression -> True
print(x > 5 and x < 10) # logical expression -> True
print(len("Python") * 2) # expression with a function call -> 12
```

Output

```
10
True
True
12
```

Statement	Expression
An instruction that performs an action.	A combination that is evaluated to a value.
Does not necessarily have a value.	Always has a value (and a type).
Examples: <code>x = 5</code> , <code>del x</code> , <code>if x > 0:</code>	Examples: <code>5</code> , <code>x + 1</code> , <code>x > 0</code> , <code>len(s)</code>
Cannot appear on the right side of <code>=</code> .	Can be part of a bigger expression or statement.

In interactive mode, typing an expression shows its value; typing a statement such as `x = 5` shows nothing.

8.3 Whitespace and indentation

Whitespace means spaces, tabs and newlines. Inside a line, extra spaces are ignored: `x=5` and `x = 5` mean the same thing (though spaces improve readability). **At the start of a line, however, whitespace is significant.** Python uses **indentation** to mark a block of statements (other languages use `{ }`).

- All statements in the same block must have the **same indentation** (4 spaces is the standard).
- A compound statement header ends with a colon, and the next line must be indented.
- Wrong indentation gives an **IndentationError**.

```
marks = 72
if marks >= 40:
    print("Pass")           # indented: belongs to the if block
    print("Well done")      # same block
print("Program ends")      # not indented: runs always
```

Output

```
Pass
Well done
Program ends
```

```
if marks >= 40:
print("Pass")           # ERROR: expected an indented block
```

Output

```
IndentationError: expected an indented block after 'if' statement on line 1
```

8.4 Comments

Comments are notes for human readers; the interpreter ignores them. They explain *why* the code does something.

- **Single-line comment:** starts with `#` and continues to the end of the line.
- **Multi-line comments:** use `#` on each line.
- **Docstring:** a string in triple quotes (`""" ... """` or `' ... '`) written as the first statement of a module or function. It documents the code and can be seen with `help()`.

```
"""
Program : Simple interest calculator
Author  : CS1139 student
"""
# Input values (hard-coded here for the example)
p, r, t = 10000, 7.5, 2
si = p * r * t / 100      # formula for simple interest
print("Simple interest:", si)
```

Output

```
Simple interest: 1500.0
```

9. Tokens

A **token** (lexical unit) is the **smallest meaningful unit** of a program. Before executing a line, the interpreter breaks it into tokens. Python has five kinds of tokens:

1. Keywords — reserved words with a fixed meaning. They cannot be used as names. Python 3.12 has 35 keywords (all lowercase except `True`, `False` and `None`):

<code>False</code>	<code>None</code>	<code>True</code>	<code>and</code>	<code>as</code>
<code>assert</code>	<code>async</code>	<code>await</code>	<code>break</code>	<code>class</code>
<code>continue</code>	<code>def</code>	<code>del</code>	<code>elif</code>	<code>else</code>
<code>except</code>	<code>finally</code>	<code>for</code>	<code>from</code>	<code>global</code>
<code>if</code>	<code>import</code>	<code>in</code>	<code>is</code>	<code>lambda</code>
<code>nonlocal</code>	<code>not</code>	<code>or</code>	<code>pass</code>	<code>raise</code>
<code>return</code>	<code>try</code>	<code>while</code>	<code>with</code>	<code>yield</code>

```
import keyword
print(len(keyword.kwlist))
print(keyword.iskeyword("while"), keyword.iskeyword("While"))
```

Output

```
35
True False
```

2. Identifiers — names given to variables, functions, modules, etc. (see Section 10).

3. Literals — fixed values written directly in the code:

Kind	Examples
Integer literals	25, -7, 0b1010 (binary), 0o17 (octal), 0xFF (hex), 1_000_000
Floating-point literals	3.14, -0.5, 2.5e3 (= 2500.0), 1e-4
Complex literals	3 + 4j
String literals	'hello', "Python", '''multi-line text'''
Boolean literals	True, False
Special literal	None (represents "no value")
Collection literals	[1, 2, 3] (list), (1, 2) (tuple), {"a": 1} (dict), {1, 2} (set)

4. Operators — symbols that perform operations: + - * // % ** == != < > <= >= and or not & | ^ ~ << >> = +=, etc. (see Part C).

5. Delimiters (punctuators) — symbols that separate or group tokens: () [] { } , : . ; @ and the quote characters.

Example: the statement below contains 9 tokens.

```
area = 3.14 * r ** 2 + side
```

Token	area	=	3.14	*	r	**	2	+	side
Kind	Identifier	Operator	Literal	Operator	Identifier	Operator	Literal	Operator	Identifier

10. Naming Objects and Variables

10.1 Rules for identifiers (must be followed)

- Can contain letters (A-Z, a-z), digits (0-9) and the underscore (_).
- **Cannot start with a digit.**
- **Cannot be a keyword.**
- No spaces or special characters such as @, \$, %, -, #.
- Python is **case-sensitive**: Total, total and TOTAL are three different names.
- Any length is allowed.

Identifier	Valid?	Reason
student_name	Valid	Letters and underscore
_count	Valid	May start with an underscore
marks2	Valid	Digit not at the start
2marks	Invalid	Starts with a digit
total marks	Invalid	Contains a space
roll-no	Invalid	Hyphen is the minus operator
for	Invalid	Keyword
For	Valid	Keywords are case-sensitive, so For is not a keyword (but it is confusing)
price\$	Invalid	\$ is not allowed

10.2 Naming conventions (should be followed, from the PEP 8 style guide)

- Use meaningful names: **average_marks** rather than **am** or **x1**.
- Variables and functions: lowercase words separated by underscores (**snake_case**), e.g. `total_salary`.
- Constants: all capitals, e.g. `PI = 3.14159`, `MAX_STUDENTS = 60`.
- Avoid using built-in names such as **print**, **input**, **list**, **sum**, **max**, **str** as variable names — doing so hides the built-in function.

```
sum = 10 + 20
print(sum([1, 2, 3]))    # TypeError: 'int' object is not callable
```

10.3 Variables and assignment

A **variable** is a **name that refers to an object** stored in memory. It is created the first time a value is assigned to it with the **assignment operator =**. In Python, a variable does not "contain" a value like a box; it is a **label attached to an object**.

```
age = 19                # simple assignment
a, b, c = 1, 2.5, "hi" # multiple assignment
x = y = z = 0           # chained assignment: all three refer to the same 0
print(age, a, b, c, x, y, z)

p, q = 5, 9
p, q = q, p              # swapping without a third variable
print("p =", p, "q =", q)

count = 10
count = count + 1        # the right side is evaluated first, then re-bound
print(count)
```

Output

```
19 1 2.5 hi 0 0 0
p = 9 q = 5
11
```

Dynamic typing: the same name can refer to objects of different types at different times.

```
value = 42
print(value, type(value))
value = "forty-two"
print(value, type(value))
```

Output

```
42 <class 'int'>
forty-two <class 'str'>
```

del removes a name; using the name afterwards raises **NameError: name 'value' is not defined**.

11. Objects and Types

11.1 Everything is an object

In Python, every piece of data — a number, a string, a list, even a function — is an **object**. Every object has three properties:

Property	Meaning	How to see it
Identity	A unique number for the object (its "address"); never changes during the object's life.	<code>id(obj)</code>
Type	The kind of object; decides what values and operations are allowed.	<code>type(obj)</code>
Value	The data the object holds.	<code>print(obj)</code>

```
a = 500
b = a                # b refers to the SAME object as a
print(type(a), a == b, id(a) == id(b))
b = b + 1            # a NEW object 501 is created; a is unchanged
print(a, b, id(a) == id(b))
```

Output

```
<class 'int'> True True
500 501 False
```

11.2 Built-in data types

Category	Type	Example	Notes
Numeric	<code>int</code>	25, -3, 10**20	Integers of unlimited size
Numeric	<code>float</code>	3.14, 2e-3	Real numbers (about 15–17 significant digits)
Numeric	<code>complex</code>	2 + 3j	j is the imaginary unit
Boolean	<code>bool</code>	True, False	A subtype of int: True = 1, False = 0
Text	<code>str</code>	"CS1139"	Sequence of characters; immutable
Sequence	<code>list</code>	[1, "a", 2.5]	Ordered, mutable
Sequence	<code>tuple</code>	(1, 2, 3)	Ordered, immutable
Mapping	<code>dict</code>	{"roll": 7}	Key-value pairs
Set	<code>set</code>	{1, 2, 3}	Unordered, unique items
None	<code>NoneType</code>	None	Absence of a value

```
print(type(7), type(7.0), type(7 + 0j))
print(type("7"), type(True), type(None))
print(2 ** 100)                                # int has no size limit
print(0.1 + 0.2, 0.1 + 0.2 == 0.3)           # floats are approximations
print(True + True, isinstance(5, int))
z = 3 + 4j
print(z.real, z.imag, abs(z))
```

Output

```
<class 'int'> <class 'float'> <class 'complex'>
<class 'str'> <class 'bool'> <class 'NoneType'>
1267650600228229401496703205376
0.30000000000000004 False
2 True
3.0 4.0 5.0
```

Example — why floating-point output sometimes looks "wrong"

```
x = 5.6
print(x)                                # looks exact
print(x * 3)                            # expected 16.8
print(x - 5)                            # expected 0.6
print(x + 0.1)                          # expected 5.7
print(x * 3 == 16.8)                    # expected True
print(format(x, ".20f"))                 # the value actually stored, to 20 decimal places
print(round(x * 3, 2), format(x - 5, ".2f")) # the usual fix for display
```

Output

```
5.6
16.799999999999997
0.5999999999999996
5.699999999999999
False
5.59999999999999964473
16.8 0.60
```

Explanation of the output:

1. **5.6 cannot be stored exactly in binary.** The integer part is easy: $5 = 101$. The fractional part 0.6 is converted by repeated multiplication by 2 (Section 2.3):

```
0.6 × 2 = 1.2  -> 1
0.2 × 2 = 0.4  -> 0
0.4 × 2 = 0.8  -> 0
0.8 × 2 = 1.6  -> 1
0.6 × 2 = 1.2  -> 1      <- 0.6 appears again, so the bits repeat forever
So 5.6 = (101.1001 1001 1001 ...) in binary
```

2. **Python stores a float in 64 bits** (IEEE 754 double precision), which keeps only about **15–17 significant decimal digits**. The endless binary fraction is cut off and rounded, so the stored value is the nearest representable number: **5.59999999999999964473...**, slightly less than 5.6 (seen in the `format(x, ".20f")` line).
3. **Why `print(x)` still shows 5.6:** when printing a float, Python shows the **shortest decimal that maps back to the same stored value**. 5.6 is that shortest decimal, so the tiny error is hidden.

4. **Why the arithmetic results show the error:** $x * 3$, $x - 5$ and $x + 0.1$ are computed from the *stored* (slightly smaller) value, and the result is rounded again to 64 bits. The combined error is now large enough that the nearest float is **not** the one closest to 16.8 (or 0.6, or 5.7), so Python has to print more digits: 16.799999999999997, 0.5999999999999996 and 5.699999999999999. For the same reason $x * 3 == 16.8$ is **False**.
5. **This is not a Python bug** — every language using binary floating point (C, Java, JavaScript) behaves the same way. Numbers such as 0.5, 0.25 or 5.75, whose fractional parts are sums of powers of 2, are stored exactly.

Rules to remember: do not compare floats with `==`; round the result (`round(value, 2)`) or format it (`format(value, ".2f")`) before displaying or comparing. For money, work in whole paise (integers) where possible.

Mutable vs immutable objects: an **immutable** object cannot be changed after it is created (int, float, bool, str, tuple). A **mutable** object can be changed in place (list, dict, set). Lists, tuples, dictionaries and sets are studied in detail in Unit IV.

11.3 Type conversion

Implicit conversion — Python converts automatically when it is safe (no data loss): `int` → `float` → `complex`.

```
print(5 + 2.0, type(5 + 2.0))    # int + float -> float
print(True + 3)                  # bool is treated as int
```

Output

```
7.0 <class 'float'>
4
```

Explicit conversion (type casting) — done by the programmer using `int()`, `float()`, `str()`, `bool()`, `complex()`.

```
print(int(9.99), int(-9.99))      # truncates toward zero
print(int("45") + 5, float("2.5"), str(100) + "1")
print(int("1010", 2))             # string in base 2 -> int
print(bool(0), bool(15), bool(""), bool("0"), bool(None))
print(round(7.456, 1), round(2.5), round(3.5)) # round-half-to-even
```

Output

```
9 -9
50 2.5 1001
10
False True False True False
7.5 2 4
```

```
int("12.5")      # ValueError: invalid literal for int() with base 10: '12.5'
"Age: " + 19     # TypeError: can only concatenate str (not "int") to str
```

Truth value (truthiness): when a value is used as a condition, 0, 0.0, "" (empty string), None, and empty collections are False; every other value is True.

11.4 Taking input from the user

input(prompt) displays the prompt, waits for the user to type something and press Enter, and **always returns a string** (type str) — even if the user types a number. The string must be converted with `int()` or `float()` before doing arithmetic.

Method 1: Standard way — one value per `input()` call

Each call to `input()` reads one line, so the user types each value on a separate line.

```
name = input("Enter your name: ")          # str, no conversion needed
age = int(input("Enter your age: "))        # str -> int
height = float(input("Enter height in feet: ")) # str -> float
print("Next year", name, "will be", age + 1)
print("Height in cm:", height * 30.48)
```

Output

```
Enter your name: Riya
Enter your age: 18
Enter height in feet: 5.6
Next year Riya will be 19
Height in cm: 170.688
```

Without conversion, `input()` returns text: if the user types 5 and 3, then `a + b` gives "53" (string joining), not 8.

Method 2: Several values on one line using `split()`

The string method `split()` breaks a line into separate strings wherever there is whitespace. With `split(",")` the line is broken at commas instead. Each piece is still a string, so each must be converted.

```
x, y = input("Enter two numbers: ").split()
print("As strings:", x, y, " x + y =", x + y)
x, y = int(x), int(y)
print("As integers: x + y =", x + y)
```

Output

```
Enter two numbers: 12 30
As strings: 12 30   x + y = 1230
As integers: x + y = 42
```

Method 3: Several values on one line using `map()`

map(function, iterable) applies the given function to **every item** of the iterable. Combined with `split()`, it converts all the values on a line in a single statement — the most common way to read several numbers at once.

Syntax

```
a, b, c = map(int, input("prompt").split())
```

How it works: `input()` returns "4 9 7" → `.split()` gives ["4", "9", "7"] → `map(int, ...)` applies `int` to each → the three integers are unpacked into `a`, `b` and `c`.

```

a, b, c = map(int, input("Enter three integers: ").split())
print("Sum =", a + b + c, " Largest =", max(a, b, c))

length, width = map(float, input("Length, width: ").split(","))
print("Area =", length * width)

nums = list(map(int, input("Enter any number of integers: ").split()))
print("Count =", len(nums), " Total =", sum(nums))

```

Output

```

Enter three integers: 4 9 7
Sum = 20  Largest = 9
Length, width: 2.5, 4.0
Area = 10.0
Enter any number of integers: 10 20 30 40 50
Count = 5  Total = 150

```

Points to remember:

- The number of variables on the left must equal the number of values typed; otherwise Python raises **ValueError** (too many / not enough values to unpack).
- Use `map(int, ...)` for whole numbers and `map(float, ...)` for decimals; a value such as 2.5 cannot be converted with `int()`.
- `list(map(...))` collects any number of values into a list (lists are studied in Unit IV).

Method	Code	User types	Values obtained
Standard (one per line)	<code>a = int(input())</code> and <code>b = int(input())</code>	4 ↵ 9 ↵	two ints
<code>split()</code>	<code>a, b = input().split()</code>	4 9 ↵	two strings "4", "9"
<code>map()</code> + <code>split()</code>	<code>a, b = map(int, input().split())</code>	4 9 ↵	two ints 4, 9
<code>map()</code> with separator	<code>a, b = map(int, input().split(","))</code>	4,9 ↵	two ints 4, 9
<code>list(map())</code>	<code>nums = list(map(int, input().split()))</code>	4 9 7 1 ↵	list [4, 9, 7, 1]

11.5 Displaying output

`print(values, sep=" ", end="\n")` displays its values separated by **sep** (a space by default) and followed by **end** (a newline by default).

```

name, age = "Riya", 18
print("Hello", name, "- age", age)
print(1, 2, 3, sep="-")
print("No newline", end=" | ")
print("same line")
print(f"{name} is {age} years old")    # f-string (see Unit III)

```

Output

```

Hello Riya - age 18
1-2-3
No newline | same line
Riya is 18 years old

```

11.6 Practice questions — Python basics

1. Identify the tokens and their kinds in: `total_marks = marks1 + marks2 * 0.5`
2. Which of these identifiers are invalid and why? `_temp`, `3rdRank`, `first-name`, `elif`, `Elif`, `net_pay$`, `rollNo2`
3. Write a program that reads the length and breadth of a rectangle and prints its area and perimeter.
4. What is the difference between a statement and an expression? Give two examples of each.
5. Predict the output: `print(type(10 / 2), type(10 // 2), type(10 * 2.0))`
6. Write a program that reads a temperature in Fahrenheit and prints it in Celsius, rounded to 2 decimal places.
7. What will `int("7") + int(7.9) + int(True)` evaluate to?
8. Why does `input() + 5` give an error, and how can it be fixed?
9. List any five immutable and three mutable built-in types.
10. Write a program that reads a number of seconds and prints it as hours, minutes and seconds (e.g. 3725 → 1 h 2 min 5 s).
11. Read three integers entered on a single line (separated by spaces) using `map()`, and print their sum and average.
12. What are the three properties of every Python object? Which of them can change?

Part C — Operators, Precedence and Associativity

12. Operators

An **operator** is a symbol (or keyword) that performs an operation on one or more values called **operands**. In `7 + 3`, `+` is the operator and 7 and 3 are operands. Operators that take one operand are **unary** (e.g. `-x`, not `x`); those that take two are **binary** (e.g. `a * b`).

12.1 Arithmetic operators

Operator	Name	Example	Result
<code>+</code>	Addition	<code>17 + 5</code>	22
<code>-</code>	Subtraction	<code>17 - 5</code>	12
<code>*</code>	Multiplication	<code>17 * 5</code>	85
<code>/</code>	True division (always gives float)	<code>17 / 5</code>	3.4
<code>//</code>	Floor division (rounds down)	<code>17 // 5</code>	3
<code>%</code>	Modulus (remainder)	<code>17 % 5</code>	2
<code>**</code>	Exponentiation (power)	<code>2 ** 5</code>	32
<code>+x, -x</code>	Unary plus / minus	<code>-(-4)</code>	4

Important points:

- `/` always returns a float: `10 / 2` gives `5.0`.
- `//` rounds **down towards minus infinity**, not towards zero: `-17 // 5` gives `-4`.

- The result of % has the **same sign as the divisor**: `-17 % 5` gives 3 and `17 % -5` gives -3. The rule is always `a == (a // b) * b + a % b`.
- If either operand is a float, `//` and `%` give a float: `7.5 // 2` is 3.0.

```
print(17 / 5, 17 // 5, 17 % 5)
print(-17 // 5, -17 % 5)
print(17 // -5, 17 % -5)
print(7.5 // 2, 7.5 % 2)
print(2 ** 10, 2 ** 0.5, 2 ** -1)
print(divmod(17, 5))           # (quotient, remainder) together
```

Output

```
3.4 3 2
-4 3
-4 -3
3.0 1.5
1024 1.4142135623730951 0.5
(3, 2)
```

Common uses of % and //: last digit of n is `n % 10`; n without its last digit is `n // 10`; n is even when `n % 2 == 0`; minutes in m seconds is `m // 60`.

12.2 Relational (comparison) operators

They compare two values and always give a **bool** (True or False).

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5.0</code>	True
<code>!=</code>	Not equal to	<code>5 != 3</code>	True
<code>></code>	Greater than	<code>3 > 7</code>	False
<code><</code>	Less than	<code>3 < 7</code>	True
<code>>=</code>	Greater than or equal to	<code>7 >= 7</code>	True
<code><=</code>	Less than or equal to	<code>8 <= 7</code>	False

- Do not confuse `=` (assignment) with `==` (comparison).
- Python allows **chained comparisons**: `1 < x < 10` means `(1 < x)` and `(x < 10)`.
- Strings are compared **character by character by their Unicode (ASCII) values**: `"A" (65) < "a" (97)`, and `"apple" < "banana"`.

```
x = 7
print(1 < x < 10, 10 < x < 20, x == 7 != 8)
print("Apple" < "apple", "cat" > "car", "10" < "9")
print(ord("A"), ord("a"), ord("0"))
```

Output

```
True False True
True True True
65 97 48
```

12.3 Logical (Boolean) operators

Operator	Meaning	Result is True when
and	Logical AND	both operands are true
or	Logical OR	at least one operand is true
not	Logical NOT	the operand is false

Truth table:

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Short-circuit evaluation: Python stops evaluating as soon as the result is known. In **A and B**, if A is false, B is never evaluated; in **A or B**, if A is true, B is never evaluated. This is useful to avoid errors, e.g. `d != 0 and n / d > 2` never divides by zero.

and / or return one of the operands, not necessarily True/False: **and** returns the first false operand (or the last one); **or** returns the first true operand (or the last one). **not** always returns a bool.

```
print(True and False, True or False, not True)
print(0 and 5, 3 and 5, 0 or 5, 3 or 5, "" or "default")
print(not 0, not "hello")
d = 0
print(d != 0 and 10 / d > 2)      # safe: division never happens
```

Output

```
False True False
0 5 5 3 default
True False
False
```

12.4 Bitwise operators

Bitwise operators work on the **binary representation** of integers, bit by bit. (This is where number systems are used directly.)

Operator	Name	Rule
&	Bitwise AND	1 only if both bits are 1
	Bitwise OR	1 if at least one bit is 1
^	Bitwise XOR	1 if the bits are different
~	Bitwise NOT	$\sim x = -(x + 1)$
<<	Left shift	$x \ll n = x \times 2^n$
>>	Right shift	$x \gg n = x // 2^n$

Worked example: a = 12 and b = 10.

```
a      = 12  = 1 1 0 0
b      = 10  = 1 0 1 0
-----
a & b   = 1 0 0 0 = 8
a | b   = 1 1 1 0 = 14
a ^ b   = 0 1 1 0 = 6
~a      = -(12 + 1) = -13
a << 2  = 1 1 0 0 0 0 = 48    (12 × 4)
a >> 2  = 1 1      = 3      (12 // 4)
```

```
a, b = 12, 10
print(a & b, a | b, a ^ b, ~a, a << 2, a >> 2)
print(bin(a & b), bin(a | b), bin(a ^ b))
print(7 & 1, 8 & 1)      # n & 1 is 1 for odd n, 0 for even n
```

Output

```
8 14 6 -13 48 3
0b1000 0b1110 0b110
1 0
```

12.5 Assignment operators

Operator	Example	Equivalent to
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
//=	x //= 3	x = x // 3
%=	x %= 3	x = x % 3
**=	x **= 3	x = x ** 3
&=, =, ^=, <<=, >>=	x <<= 1	x = x << 1

Python has **no ++ or -- operators**; use `x += 1` and `x -= 1`. (Writing `++x` is legal but simply means `+(+x)`, which does not change x.)

```
x = 20
x += 5; print(x)
x /= 4; print(x)
x **= 3; print(x)
x %= 10; print(x)
```

Output

```
25
6
216
6
```

13. Precedence and Associativity

Precedence decides **which operator is applied first** when an expression contains different operators (like BODMAS in mathematics). **Associativity** decides the **order in which operators of the same precedence** are applied: left-to-right or right-to-left.

13.1 Precedence table (highest to lowest)

Level	Operator(s)	Description	Associativity
1	()	Parentheses (grouping)	—
2	x[i], f(args), obj.attr	Indexing, function call, attribute	Left to right
3	**	Exponentiation	Right to left
4	+x, -x, ~x	Unary plus, unary minus, bitwise NOT	Right to left
5	*, /, //, %	Multiplication, division, floor division, modulus	Left to right
6	+, -	Addition, subtraction	Left to right
7	<<, >>	Bitwise shifts	Left to right
8	&	Bitwise AND	Left to right
9	^	Bitwise XOR	Left to right
10		Bitwise OR	Left to right
11	==, !=, >, <, >=, <=	Comparisons (relational)	Chained (see below)
12	not	Logical NOT	Right to left
13	and	Logical AND	Left to right
14	or	Logical OR	Left to right
15	if - else	Conditional expression	Right to left
16	lambda	Lambda expression	—
17	:=	Assignment expression ("walrus")	Right to left

Points to remember:

- **** is right-associative:** `2 ** 3 ** 2` = `2 ** (3 ** 2)` = `2 ** 9` = 512 (not 64).
- **** binds more tightly than a unary minus on its left:** `-3 ** 2` = `-(3 ** 2)` = -9. But on its right, `2 ** -1` = 0.5.
- **Comparison operators are chained**, not grouped: `a < b < c` means `a < b` and `b < c`. Hence `5 > 3 > 1` is True even though `(5 > 3) > 1` is False.
- **not has lower precedence than comparisons:** `not a == b` means `not (a == b)`.
- **and is evaluated before or:** `a or b and c` means `a or (b and c)`.
- Arithmetic is done before comparison, and comparison before logical operators.
- The assignment `=` is a statement, not an operator, so it has no precedence; the right side is always evaluated completely first.
- **When in doubt, use parentheses** — they make the intention clear.

13.2 Solved examples — evaluating expressions

Example 1: `10 + 20 * 3 / 5 - 2`

```
= 10 + 60 / 5 - 2      (* first, left to right with /)
= 10 + 12.0 - 2        (/ gives a float)
= 22.0 - 2
= 20.0
```

Example 2: `2 ** 3 ** 2 // 10 % 7`

```
= 2 ** 9 // 10 % 7      (** is right to left: 3 ** 2 = 9 first)
= 512 // 10 % 7
= 51 % 7                 (// and % have equal precedence: left to right)
= 2
```

Example 3: `-2 ** 4 + 17 % -5`

```
= -(2 ** 4) + (17 % -5)
= -16 + (-3)             (result of % takes the sign of the divisor)
= -19
```

Example 4: `100 // 7 * 7 + 100 % 7`

```
= 14 * 7 + 2
= 98 + 2
= 100                    (always true: (a // b) * b + a % b == a)
```

Example 5: `5 > 3 and 4 < 2 or not 7 == 7`

```
= (5 > 3) and (4 < 2) or (not (7 == 7))    comparisons first
= True and False or not True
= True and False or False                  not before and
= False or False                          and before or
= False
```

Example 6: `3 + 4 > 6 == True`

```
Chained comparison: (3 + 4 > 6) and (6 == True)
= (7 > 6) and (6 == 1)
= True and False
= False
```

Example 7: `6 | 3 & 5 ^ 2`

```
& first : 3 & 5 = 011 & 101 = 001 = 1
^ next  : 1 ^ 2 = 01 ^ 10 = 11 = 3
| last   : 6 | 3 = 110 | 011 = 111 = 7
```

Example 8: `1 + 2 << 3 >> 1`

```
= (1 + 2) << 3 >> 1    (+ is higher than shifts)
= 3 << 3 >> 1          (shifts left to right)
= 24 >> 1
= 12
```

Example 9: 0 or 7 and 8 or 9

```
= 0 or (7 and 8) or 9
= 0 or 8 or 9          (7 is true, so "and" returns the second operand)
= 8                    ("or" returns the first true operand)
```

Example 10: x = 5 then x += x * 2 - 3 ** 2 // 4

```
Right side first: x * 2 - 3 ** 2 // 4
                  = 10 - 9 // 4
                  = 10 - 2 = 8
x = x + 8 = 13
```

Verification in Python:

```
print(10 + 20 * 3 / 5 - 2)
print(2 ** 3 ** 2 // 10 % 7)
print(-2 ** 4 + 17 % -5)
print(100 // 7 * 7 + 100 % 7)
print(5 > 3 and 4 < 2 or not 7 == 7)
print(3 + 4 > 6 == True)
print(6 | 3 & 5 ^ 2)
print(1 + 2 << 3 >> 1)
print(0 or 7 and 8 or 9)
x = 5
x += x * 2 - 3 ** 2 // 4
print(x)
```

Output

```
20.0
2
-19
100
False
False
7
12
8
13
```

13.3 Practice questions — operators

Evaluate by hand, showing the order of operations, then check in Python:

1. `25 - 4 * 3 + 18 / 6 ** 2`
2. `3 ** 2 ** 2 % 10`
3. `-23 // 4`, `-23 % 4` and `23 % -4`
4. `7 + 3 * 2 > 12` and `not 5 % 2 == 0`
5. `4 > 3 > 2` and `(4 > 3) > 2`
6. `15 & 9 | 4 ^ 6`
7. `~7 + (40 >> 3) - (3 << 2)`
8. `"" or 0 or [] or "Python" and 10`
9. `10 - 2 ** 3 * -1 + 9 // 2`

10. `not 4 - 4 or 2 * 3 == 6.0`

11. `17 % 5 * 3 ** 2 // 4`

12. After `a = 7` and `a *= 2 + 3`, what is `a`? (Hint: the whole right side is evaluated first.)

Answers: 1) 13.5 2) 1 3) -6, 1, -1 4) True 5) False 6) 11 7) -15 8) 10 9) 22 10) True 11) 4 12) 35

Part D — Conditional Statements (Unit II)

14. Control Flow

By default, statements are executed one after another from top to bottom — this is **sequential** flow. Real programs also need to **choose** between actions and to **repeat** actions. Python provides three kinds of control structures:

Control structure	Purpose	Python statements
Sequence	Execute statements one after another.	(normal flow)
Selection (decision)	Choose which block to execute based on a condition.	if, if-else, if-elif-else
Repetition (iteration)	Execute a block repeatedly.	for, while

A **condition** is any expression whose value is treated as True or False — usually a relational or logical expression such as `marks >= 40` or `age > 18` and `has_id`.

15. The Selection Statements

15.1 The if statement

Executes a block **only if** the condition is True; otherwise the block is skipped.

Syntax

```
if condition:
    statement(s)           # executed only when condition is True
```

```
num = int(input("Enter a number: "))
if num < 0:
    num = -num
    print("Converted to positive")
print("Absolute value:", num)
```

Output

```
Enter a number: -8
Converted to positive
Absolute value: 8
```

15.2 The if-else statement

Chooses between **two** alternative blocks: exactly one of them is executed.

Syntax

```
if condition:
    block-1           # when condition is True
else:
    block-2           # when condition is False
```

```
n = int(input("Enter a number: "))
if n % 2 == 0:
    print(n, "is even")
else:
    print(n, "is odd")
```

Output

```
Enter a number: 36
36 is even
```

15.3 The if-elif-else ladder

Used when there are **more than two** alternatives. Conditions are checked **from top to bottom**; the block of the **first true condition** is executed and the rest are skipped. The final else (optional) runs if no condition is true.

Syntax

```
if condition-1:
    block-1
elif condition-2:
    block-2
elif condition-3:
    block-3
else:
    block-n           # when none of the conditions is True
```

```
n = int(input("Enter a number: "))
if n > 0:
    print("Positive")
elif n < 0:
    print("Negative")
else:
    print("Zero")
```

Output

```
Enter a number: 0
Zero
```

Order matters in a ladder. If marks ≥ 40 is tested before marks ≥ 80 , a student with 95 marks gets the grade meant for 40 because the first true condition wins. Always test the most restrictive condition first.

15.4 Nested if statements

An if (or if-else) placed **inside the block of another** if or else is called a **nested if**. The inner condition is tested only when the outer condition allows it.

Syntax

```
if condition-1:
    if condition-2:
        block-A          # condition-1 True and condition-2 True
    else:
        block-B          # condition-1 True and condition-2 False
else:
    block-C              # condition-1 False
```

```
age = int(input("Enter age: "))
if age >= 18:
    has_id = input("Do you have a voter ID (yes/no)? ")
    if has_id == "yes":
        print("You can vote.")
    else:
        print("Eligible, but get a voter ID first.")
else:
    print("Not eligible. Wait", 18 - age, "more year(s).")
```

Output

```
Enter age: 21
Do you have a voter ID (yes/no)? yes
You can vote.
```

A nested if can often be rewritten with a logical operator (if a and b:), but nesting is clearer when the inner test only makes sense after the outer test has passed (as with asking for the ID above).

15.5 The conditional expression (ternary operator)

A compact one-line form for choosing between two **values**:

Syntax

```
value_if_true if condition else value_if_false
```

```
a, b = 14, 9
larger = a if a > b else b
print("Larger:", larger)
print("Even" if a % 2 == 0 else "Odd")
```

Output

```
Larger: 14
Even
```

15.6 Common mistakes

- Missing colon at the end of the if / elif / else line → `SyntaxError`.
- Using `=` instead of `==` in a condition → `SyntaxError`.
- Inconsistent indentation → `IndentationError`, or wrong logic when a statement falls outside the block.
- Writing `if x == 1 or 2:` — this is always `True` because it means `(x == 1) or 2`, and `2` is `truthy`. Write `if x == 1 or x == 2:`.
- Comparing the string returned by `input()` with a number: `input() == 5` is always `False`. Convert first.

- An elif or else must have a matching if at the same indentation level.

16. Solved Examples — Selection

Example 1: Leap year. A year is a leap year if it is divisible by 400, or divisible by 4 but not by 100.

```
year = int(input("Enter a year: "))
if year % 400 == 0:
    print(year, "is a leap year")
elif year % 100 == 0:
    print(year, "is not a leap year")
elif year % 4 == 0:
    print(year, "is a leap year")
else:
    print(year, "is not a leap year")
```

Output (run 1)

```
Enter a year: 2024
2024 is a leap year
```

Output (run 2)

```
Enter a year: 1900
1900 is not a leap year
```

Output (run 3)

```
Enter a year: 2000
2000 is a leap year
```

The same logic in a single condition: `if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:`

Example 2: Parking fee with input validation. A parking lot charges ₹30 for the first 2 hours and ₹20 for every additional hour (a part of an hour counts as a full hour), with a maximum charge of ₹200 per day. Parking time must be more than 0 and at most 24 hours.

```
hours = float(input("Hours parked: "))
if hours <= 0 or hours > 24:
    print("Invalid parking time")
else:
    full_hours = int(hours)
    if hours > full_hours:          # a part of an hour counts as a full hour
        full_hours = full_hours + 1
    if full_hours <= 2:
        fee = 30
    else:
        fee = 30 + (full_hours - 2) * 20
    if fee > 200:
        fee = 200
    print("Parking fee: Rs", fee)
```

Output (run 1)

```
Hours parked: 5.5
Parking fee: Rs 110
```

Output (run 2)

Hours parked: 13
Parking fee: Rs 200

Output (run 3)

Hours parked: 26
Invalid parking time

Example 3: Triangle validity and type from three sides. Three lengths form a triangle only if the sum of any two sides is greater than the third.

```
a = float(input("Side a: "))
b = float(input("Side b: "))
c = float(input("Side c: "))
if a + b > c and b + c > a and a + c > b:
    if a == b == c:
        print("Equilateral triangle")
    elif a == b or b == c or a == c:
        print("Isosceles triangle")
    else:
        print("Scalene triangle")
else:
    print("These sides do not form a triangle")
```

Output (run 1)

Side a: 5
Side b: 5
Side c: 8
Isosceles triangle

Output (run 2)

Side a: 3
Side b: 4
Side c: 9
These sides do not form a triangle

Example 4: Roots of a quadratic equation $ax^2 + bx + c = 0$, using the discriminant $D = b^2 - 4ac$.

```
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))
if a == 0:
    print("Not a quadratic equation")
else:
    d = b * b - 4 * a * c
    if d > 0:
        r1 = (-b + d ** 0.5) / (2 * a)
        r2 = (-b - d ** 0.5) / (2 * a)
        print("Real and distinct roots:", r1, r2)
    elif d == 0:
        print("Real and equal roots:", -b / (2 * a))
    else:
        real = -b / (2 * a)
        imag = (-d) ** 0.5 / (2 * a)
        print("Complex roots:", real, "+", imag, "i and", real, "-", imag, "i")
```

Output (run 1)

```
a: 1
b: -5
c: 6
Real and distinct roots: 3.0 2.0
```

Output (run 2)

```
a: 1
b: 2
c: 5
Complex roots: -1.0 + 2.0 i and -1.0 - 2.0 i
```

Example 5: Electricity bill with slabs. First 100 units at ₹4.50, next 100 at ₹6.00, next 100 at ₹7.50, above 300 at ₹9.00 per unit. A fixed charge of ₹75 is added.

```
units = int(input("Units consumed: "))
if units <= 100:
    amount = units * 4.50
elif units <= 200:
    amount = 100 * 4.50 + (units - 100) * 6.00
elif units <= 300:
    amount = 100 * 4.50 + 100 * 6.00 + (units - 200) * 7.50
else:
    amount = 100 * 4.50 + 100 * 6.00 + 100 * 7.50 + (units - 300) * 9.00
bill = amount + 75
print("Energy charge: Rs", amount)
print("Total bill : Rs", bill)
```

Output

```
Units consumed: 265
Energy charge: Rs 1537.5
Total bill : Rs 1612.5
```

Example 6: Classify a character as uppercase letter, lowercase letter, digit or special character.

```
ch = input("Enter a single character: ")
if "A" <= ch <= "Z":
    print(ch, "is an uppercase letter")
elif "a" <= ch <= "z":
    print(ch, "is a lowercase letter")
elif "0" <= ch <= "9":
    print(ch, "is a digit")
else:
    print(ch, "is a special character")
```

Output (run 1)

```
Enter a single character: G
G is an uppercase letter
```

Output (run 2)

```
Enter a single character: 7
7 is a digit
```

Output (run 3)

Enter a single character: #
is a special character

Example 7: Quadrant of a point (x, y).

```
x = float(input("x: "))
y = float(input("y: "))
if x == 0 and y == 0:
    print("Origin")
elif x == 0:
    print("On the Y-axis")
elif y == 0:
    print("On the X-axis")
elif x > 0:
    if y > 0:
        print("Quadrant I")
    else:
        print("Quadrant IV")
else:
    if y > 0:
        print("Quadrant II")
    else:
        print("Quadrant III")
```

Output (run 1)

x: -3
y: 4
Quadrant II

Output (run 2)

x: 0
y: 5
On the Y-axis

Example 8: Simple calculator using an operator entered by the user.

```
a = float(input("First number: "))
op = input("Operator (+ - * / %): ")
b = float(input("Second number: "))
if op == "+":
    print("Result:", a + b)
elif op == "-":
    print("Result:", a - b)
elif op == "*":
    print("Result:", a * b)
elif op == "/" or op == "%":
    if b == 0:
        print("Error: division by zero")
    elif op == "/":
        print("Result:", a / b)
    else:
        print("Result:", a % b)
else:
    print("Unknown operator")
```

Output (run 1)

```
First number: 45
Operator (+ - * / %): /
Second number: 6
Result: 7.5
```

Output (run 2)

```
First number: 9
Operator (+ - * / %): %
Second number: 0
Error: division by zero
```

Example 9: Movie ticket price — base price ₹250; children below 12 and seniors aged 60+ get 40% off; students (age 12–25 with a student card) get 20% off; weekend shows cost ₹50 extra for everyone.

```
age = int(input("Age: "))
student = input("Student card (y/n)? ")
weekend = input("Weekend show (y/n)? ")
price = 250
if age < 12 or age >= 60:
    price = price * 0.60
elif age <= 25 and student == "y":
    price = price * 0.80
if weekend == "y":
    price = price + 50
print("Ticket price: Rs", price)
```

Output

```
Age: 19
Student card (y/n)? y
Weekend show (y/n)? n
Ticket price: Rs 200.0
```

Note that the weekend check is a **separate if**, not an **elif**, because it must be applied independently of the discount.

Example 10: Number of days in a month.

```
month = int(input("Month (1-12): "))
year = int(input("Year: "))
if month < 1 or month > 12:
    print("Invalid month")
elif month == 2:
    if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
        days = 29
    else:
        days = 28
    print("Days:", days)
elif month == 4 or month == 6 or month == 9 or month == 11:
    print("Days: 30")
else:
    print("Days: 31")
```

Output (run 1)

Month (1-12): 2
Year: 2028
Days: 29

Output (run 2)

Month (1-12): 11
Year: 2026
Days: 30

Example 11: Largest of three numbers (compare with Flowchart 2) — first with nested if, then with logical operators.

```
a = int(input("a: "))
b = int(input("b: "))
c = int(input("c: "))
# Method 1: nested if
if a > b:
    if a > c:
        largest = a
    else:
        largest = c
else:
    if b > c:
        largest = b
    else:
        largest = c
print("Largest (nested if):", largest)
# Method 2: logical operators
if a >= b and a >= c:
    largest = a
elif b >= c:
    largest = b
else:
    largest = c
print("Largest (and):", largest)
```

Output

a: 12
b: 45
c: 31
Largest (nested if): 45
Largest (and): 45

Example 12: Income tax (simplified slabs). No tax up to ₹4,00,000; 5% on income from ₹4,00,001 to ₹8,00,000; 10% on income above ₹8,00,000 up to ₹12,00,000; 15% above ₹12,00,000. (Illustrative slabs only.)

```
income = float(input("Annual income: "))
if income <= 400000:
    tax = 0
elif income <= 800000:
    tax = (income - 400000) * 0.05
elif income <= 1200000:
    tax = 400000 * 0.05 + (income - 800000) * 0.10
else:
    tax = 400000 * 0.05 + 400000 * 0.10 + (income - 1200000) * 0.15
print("Tax payable: Rs", tax)
```

Output

```
Annual income: 1050000
Tax payable: Rs 45000.0
```

17. Practice Problems — Selection

1. Read a number and print whether it is divisible by both 3 and 7, by only one of them, or by neither.
2. Read three angles and check whether they form a valid triangle; if valid, print whether it is acute, right or obtuse angled.
3. Read a character and check whether it is a vowel or a consonant (handle both upper and lower case, and reject non-letters).
4. Read the cost price and selling price of an item and print whether there is a profit or a loss and by how much (in amount and percentage).
5. Read a three-digit number and check whether all its digits are the same, exactly two are the same, or all are different.
6. Read the day number (1-7) and print the name of the day, and whether it is a weekday or weekend.
7. Read the height (in metres) and weight (in kg) of a person, compute $BMI = \text{weight} / \text{height}^2$, and print the category: underweight (< 18.5), normal ($18.5-24.9$), overweight ($25-29.9$), obese (≥ 30).
8. Read a year and a month number and print the season (e.g. Dec-Feb: Winter, Mar-May: Summer, Jun-Sep: Monsoon, Oct-Nov: Post-monsoon), rejecting invalid months.
9. A shop gives a discount of 5% on purchases above ₹2000, 10% above ₹5000 and 15% above ₹10000; members get an extra 2% off on the discounted price. Read the amount and membership status and print the final amount.
10. Read the coordinates of the centre and the radius of a circle and a point (x, y); print whether the point lies inside, on, or outside the circle.
11. Read two times in 24-hour format (hours and minutes) and print which one is earlier, or that they are equal.
12. Read a person's age, gender and marital status and print the insurance premium category using nested if: married drivers get category A; unmarried male drivers above 30 get category B; unmarried female drivers above 25 get category C; everyone else gets category D.

Part E — Repetition (Loops)

18. Why Loops?

A **loop** (iteration) repeats a block of statements as long as required. Without loops, printing the numbers 1 to 1000 would need 1000 print statements. Every loop has three parts:

- **Initialisation:** giving the loop variable a starting value ($i = 1$).
- **Condition (test):** deciding whether to run the body again ($i \leq 10$).
- **Update:** changing the loop variable so that the loop eventually stops ($i += 1$).

Python has two loop statements: **for** (used when the items or the number of repetitions are known in advance) and **while** (used when repetition depends on a condition).

19. The for Loop

19.1 Syntax

The for loop takes each item of a **sequence** (range, string, list, ...) one by one, assigns it to the loop variable, and executes the body once for each item.

Syntax

```
for variable in sequence:
    body
```

19.2 The range() function

range() generates a sequence of integers. The **stop value is never included**.

Call	Generates	Explanation
<code>range(5)</code>	0, 1, 2, 3, 4	start = 0, step = 1 by default
<code>range(2, 7)</code>	2, 3, 4, 5, 6	from start up to stop - 1
<code>range(1, 10, 2)</code>	1, 3, 5, 7, 9	step of 2
<code>range(10, 0, -3)</code>	10, 7, 4, 1	negative step counts down
<code>range(5, 1)</code>	(empty)	start $\geq$ stop with a positive step

```
for i in range(5):
    print(i, end=" ")
print()
for i in range(10, 0, -3):
    print(i, end=" ")
print()
for ch in "LOOP":
    print(ch, end="-")
print()
print(list(range(2, 20, 4)))
```

Output

```
0 1 2 3 4
10 7 4 1
L-0-0-P-
[2, 6, 10, 14, 18]
```

19.3 Example — sum and average of N numbers

```
n = int(input("How many numbers? "))
total = 0
for i in range(1, n + 1):
    x = float(input("Enter number " + str(i) + ": "))
    total += x
print("Sum =", total, " Average =", total / n)
```

Output

```
How many numbers? 4
Enter number 1: 12
Enter number 2: 7
Enter number 3: 20
Enter number 4: 9
Sum = 48.0  Average = 12.0
```

20. The while Loop

20.1 Syntax

The while loop **checks a condition before every iteration** and repeats the body as long as the condition is True. If the condition is False at the start, the body never runs.

Syntax

```
initialisation
while condition:
    body
    update
```

```
i = 1
while i <= 5:
    print("Iteration", i)
    i += 1
print("After the loop, i =", i)
```

Output

```
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5
After the loop, i = 6
```

20.2 Counter-controlled vs sentinel-controlled loops

A **counter-controlled** loop runs a known number of times (as above). A **sentinel-controlled** loop keeps running until a special value (the **sentinel**) is entered — the number of repetitions is not known in advance.

```
total = 0
count = 0
num = int(input("Enter a number (0 to stop): "))
while num != 0:                # 0 is the sentinel
    total += num
    count += 1
    num = int(input("Enter a number (0 to stop): "))
print("Numbers entered:", count, " Sum:", total)
```

Output

```
Enter a number (0 to stop): 15
Enter a number (0 to stop): -4
Enter a number (0 to stop): 22
Enter a number (0 to stop): 8
Enter a number (0 to stop): 0
Numbers entered: 4 Sum: 41
```

20.3 for vs while

for loop	while loop
Iterates over a sequence or a range.	Repeats as long as a condition is True.
Number of iterations is known in advance.	Number of iterations may be unknown.
Initialisation and update are automatic.	Initialisation and update must be written by the programmer.
Cannot become infinite by forgetting the update (for a range).	Forgetting the update gives an infinite loop.
Example: print a table, process each character.	Example: process digits of a number, read until a sentinel.

Every for loop over a range can be rewritten as a while loop, and vice versa.

21. break, continue and pass

Statement	Effect
<code>break</code>	Immediately exits the innermost loop; control goes to the statement after the loop.
<code>continue</code>	Skips the rest of the current iteration and goes to the next iteration (the condition is checked again).
<code>pass</code>	Does nothing ; used as a placeholder where a statement is syntactically required.

```

print("break    :", end=" ")
for i in range(1, 11):
    if i == 6:
        break
    print(i, end=" ")
print()
print("continue:", end=" ")
for i in range(1, 11):
    if i % 3 == 0:
        continue
    print(i, end=" ")
print()
for i in range(3):
    pass                # to be written later
print("pass does nothing; i =", i)

```

Output

```

break    : 1 2 3 4 5
continue: 1 2 4 5 7 8 10
pass does nothing; i = 2

```

The else clause of a loop: a loop may have an else block, which runs **only if the loop ends normally** (not through break). It is handy for search problems — for example, finding the first number in a range that is divisible by both 6 and 8.

```

a = int(input("Start: "))
b = int(input("End: "))
for num in range(a, b + 1):
    if num % 6 == 0 and num % 8 == 0:
        print("First number divisible by 6 and 8:", num)
        break
else:
    print("No number between", a, "and", b, "is divisible by 6 and 8")

```

Output (run 1)

```

Start: 30
End: 60
First number divisible by 6 and 8: 48

```

Output (run 2)

```

Start: 50
End: 70
No number between 50 and 70 is divisible by 6 and 8

```

22. Infinite Loops

A loop whose condition never becomes False is an **infinite loop**.

Accidental infinite loops usually happen because the update is missing or wrong:

```

i = 1
while i <= 10:
    print(i)                # i is never increased -> prints 1 forever

```

```
i = 10
while i != 0:
    i -= 3          # 10, 7, 4, 1, -2, ... never exactly 0
```

To stop a running program, press **Ctrl + C** (or the stop button in Spyder).

Intentional infinite loops use **while True:** with a **break** inside — useful when the exit point is in the middle of the body. Example: find the smallest n for which $1 + 2 + \dots + n$ exceeds a given limit.

```
limit = int(input("Limit: "))
n = 0
total = 0
while True:
    n += 1
    total += n
    if total > limit:
        break
print("Smallest n =", n, "and the sum =", total)
```

Output

```
Limit: 100
Smallest n = 14 and the sum = 105
```

23. Solved Examples — Loops

Example 1: Sum of even and odd numbers separately from 1 to N.

```
n = int(input("N: "))
even_sum = odd_sum = 0
for i in range(1, n + 1):
    if i % 2 == 0:
        even_sum += i
    else:
        odd_sum += i
print("Sum of even numbers:", even_sum)
print("Sum of odd numbers :", odd_sum)
```

Output

```
N: 10
Sum of even numbers: 30
Sum of odd numbers : 25
```

Example 2: Count the digits and find the product of the digits of a number.

```
n = int(input("Enter a positive integer: "))
temp = n
count = 0
product = 1
while temp > 0:
    digit = temp % 10
    product *= digit
    count += 1
    temp //= 10
print(n, "has", count, "digits; product of digits =", product)
```

Output

Enter a positive integer: 47125
47125 has 5 digits; product of digits = 280

Example 3: Palindrome number (the number equals its reverse).

```
n = int(input("Enter a number: "))
original, rev = n, 0
while n > 0:
    rev = rev * 10 + n % 10
    n //= 10
if rev == original:
    print(original, "is a palindrome")
else:
    print(original, "is not a palindrome")
```

Output (run 1)

Enter a number: 12321
12321 is a palindrome

Output (run 2)

Enter a number: 12345
12345 is not a palindrome

Example 4: Harshad (Niven) number — a number that is divisible by the sum of its own digits (e.g. 156: $1 + 5 + 6 = 12$ and 156 is divisible by 12).

```
n = int(input("Enter a number: "))
temp, digit_sum = n, 0
while temp > 0:
    digit_sum += temp % 10
    temp //= 10
if n % digit_sum == 0:
    print(n, "is a Harshad number; digit sum =", digit_sum)
else:
    print(n, "is not a Harshad number; digit sum =", digit_sum)
```

Output (run 1)

Enter a number: 156
156 is a Harshad number; digit sum = 12

Output (run 2)

Enter a number: 157
157 is not a Harshad number; digit sum = 13

Example 5: Fibonacci series — first N terms.

```
n = int(input("How many terms? "))
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
```

Output

How many terms? 12
0 1 1 2 3 5 8 13 21 34 55 89

Example 6: GCD and LCM using Euclid's algorithm (compare with Algorithm Example 3).

```
a = int(input("a: "))
b = int(input("b: "))
x, y = a, b
while y != 0:
    x, y = y, x % y
gcd = x
lcm = a * b // gcd
print("GCD =", gcd, " LCM =", lcm)
```

Output

a: 84
b: 36
GCD = 12 LCM = 252

Example 7: Perfect number — equal to the sum of its proper divisors ($28 = 1 + 2 + 4 + 7 + 14$).

```
n = int(input("Enter a number: "))
s = 0
for d in range(1, n // 2 + 1):
    if n % d == 0:
        s += d
print(n, "is a perfect number" if s == n else "is not a perfect number")
```

Output (run 1)

Enter a number: 28
28 is a perfect number

Output (run 2)

Enter a number: 36
36 is not a perfect number

Example 8: Power x^n without using $$** (repeated multiplication; also handles negative n).

```
x = float(input("Base x: "))
n = int(input("Exponent n: "))
result = 1
for i in range(abs(n)):
    result *= x
if n < 0:
    result = 1 / result
print(x, "^", n, "=", result)
```

Output (run 1)

Base x: 2
Exponent n: 10
2.0 ^ 10 = 1024.0

Output (run 2)

Base x: 5
Exponent n: -2
 $5.0^{-2} = 0.04$

Example 9: Decimal to binary conversion (repeated division by 2, as in Section 2.3).

```
n = int(input("Decimal number: "))
if n == 0:
    binary = "0"
else:
    binary = ""
    temp = n
    while temp > 0:
        binary = str(temp % 2) + binary    # new bit goes to the LEFT
        temp //= 2
print("Binary of", n, "is", binary, "| check with bin():", bin(n))
```

Output (run 1)

Decimal number: 45
Binary of 45 is 101101 | check with bin(): 0b101101

Output (run 2)

Decimal number: 156
Binary of 156 is 10011100 | check with bin(): 0b10011100

Example 10: Sum of the series $1 - 1/2 + 1/3 - 1/4 + \dots$ up to N terms.

```
n = int(input("Number of terms: "))
total = 0
sign = 1
for i in range(1, n + 1):
    total += sign / i
    sign = -sign    # alternate + and -
print("Sum =", round(total, 5))
```

Output

Number of terms: 6
Sum = 0.61667

Example 11: Largest, smallest and count of numbers entered until the user types a negative number (sentinel).

```
num = int(input("Enter a number (negative to stop): "))
if num < 0:
    print("No numbers entered")
else:
    largest = smallest = num
    count = 0
    while num >= 0:
        count += 1
        if num > largest:
            largest = num
        if num < smallest:
            smallest = num
        num = int(input("Enter a number (negative to stop): "))
    print("Count:", count, " Largest:", largest, " Smallest:", smallest)
```

Output

```
Enter a number (negative to stop): 34
Enter a number (negative to stop): 81
Enter a number (negative to stop): 7
Enter a number (negative to stop): 56
Enter a number (negative to stop): -1
Count: 4 Largest: 81 Smallest: 7
```

Example 12: Years needed to double an investment at a given annual compound interest rate — the number of iterations is not known in advance, so a while loop is used.

```
amount = float(input("Initial amount: "))
rate = float(input("Annual interest rate (%): "))
target = 2 * amount
years = 0
while amount < target:
    amount = amount + amount * rate / 100
    years += 1
print("Money doubles in", years, "years; amount =", round(amount, 2))
```

Output

```
Initial amount: 10000
Annual interest rate (%): 8
Money doubles in 10 years; amount = 21589.25
```

24. Practice Problems — Loops

1. Print all numbers between 1 and 100 that are divisible by 7 but not by 5.
2. Read N and print the sum of the squares $1^2 + 2^2 + \dots + N^2$.
3. Read a number and print the sum of its first and last digits.
4. Read a number and check whether it is a strong number (sum of factorials of its digits equals the number, e.g. $145 = 1! + 4! + 5!$).
5. Print the multiplication table of N in the reverse order ($N \times 10$ down to $N \times 1$) using a while loop.
6. Read two numbers a and b and print all palindrome numbers between a and b.

7. Read a binary number (as an integer, e.g. 101101) and convert it to decimal using a loop.
8. Read numbers until the user enters 0 and print the average of only the positive numbers.
9. Print the sum of the series $1 + x + x^2/2! + x^3/3! + \dots$ up to N terms for given x and N.
10. Read a number and print how many times a given digit occurs in it (e.g. digit 3 occurs 2 times in 53731).
11. Print all numbers between 1 and 1000 whose digits add up to 10, and count them.
12. Implement a menu-driven program using while True: 1) square of a number, 2) cube of a number, 3) exit. Keep showing the menu until the user chooses exit.

25. Nested Loops

25.1 Concept

A loop placed **inside the body of another loop** is a **nested loop**. For **every single iteration of the outer loop, the inner loop runs completely**. If the outer loop runs m times and the inner loop runs n times per outer iteration, the inner body executes $m \times n$ times.

```
for i in range(1, 4):           # outer loop: 3 times
    for j in range(1, 3):       # inner loop: 2 times for EACH i
        print("i =", i, " j =", j)
```

Output

```
i = 1  j = 1
i = 1  j = 2
i = 2  j = 1
i = 2  j = 2
i = 3  j = 1
i = 3  j = 2
```

In patterns, the **outer loop usually controls the rows** and the **inner loop(s) control what is printed in each row** (the columns). `print(..., end="")` keeps the output on the same line and a plain `print()` moves to the next line after each row.

A **break** inside the inner loop exits only the inner loop; the outer loop continues.

25.2 All four combinations

Any loop can be nested inside any other. The four combinations are shown below with the same idea (or a natural use) so that they can be compared.

(a) for inside for — multiplication tables of 2 to 4 side by side:

```
for i in range(1, 6):           # rows: multiplier 1..5
    for t in range(2, 5):       # columns: tables of 2, 3, 4
        print(f"{t} x {i} = {t * i:2}", end="    ")
    print()
```

Output

```
2 x 1 =  2    3 x 1 =  3    4 x 1 =  4
2 x 2 =  4    3 x 2 =  6    4 x 2 =  8
2 x 3 =  6    3 x 3 =  9    4 x 3 = 12
2 x 4 =  8    3 x 4 = 12    4 x 4 = 16
2 x 5 = 10    3 x 5 = 15    4 x 5 = 20
```

(b) while inside for — for each number in a range, find its digit sum with a while loop:

```
for n in range(115, 121):
    s, temp = 0, n
    while temp > 0:             # inner while: process digits of n
        s += temp % 10
        temp //= 10
    print("Digit sum of", n, "=", s)
```

Output

Digit sum of 115 = 7
Digit sum of 116 = 8
Digit sum of 117 = 9
Digit sum of 118 = 10
Digit sum of 119 = 11
Digit sum of 120 = 3

(c) **for inside while** — print a rectangle of numbers row by row:

```
row = 1
while row <= 4:                # outer while: 4 rows
    for col in range(1, 6):    # inner for: 5 columns
        print(f"{row * col:4}", end="")
    print()
    row += 1
```

Output

```
1  2  3  4  5
2  4  6  8 10
3  6  9 12 15
4  8 12 16 20
```

(d) **while inside while** — print all pairs (i, j) with $i \leq j$ whose product is 36:

```
i = 1
while i <= 36:
    j = i
    while j <= 36:            # inner while: try every j from i
        if i * j == 36:
            print(i, "x", j, "= 36")
        j += 1
    i += 1
```

Output

```
1 x 36 = 36
2 x 18 = 36
3 x 12 = 36
4 x 9 = 36
6 x 6 = 36
```

Combination	When it is natural
for – for	Both counts are known (grids, tables, most patterns).
for – while	Known number of items, each needing an unknown number of steps (digits of each number).
while – for	Unknown number of rounds (menus, sentinel input), each doing a fixed task.
while – while	Both counts depend on conditions (searching for pairs or values that satisfy a condition).

26. Patterns

26.1 How to approach a pattern

1. Count the **rows**; the outer loop runs once per row (row number i).
2. For each row, work out **how many spaces** and **how many symbols** are printed, as a formula in i and n .
3. Decide **what** is printed: a fixed symbol (*), the row number (i), the column number (j), a running counter, or a letter (`chr(64 + j)` gives A, B, C, ...).
4. Write an inner loop (or string repetition) for the spaces, then for the symbols, and finish the row with `print()`.

chr() and **ord()**: `ord("A")` is 65 and `chr(65)` is "A". So `chr(65 + k)` gives the k -th letter after A. String repetition such as `" " * 3` gives three spaces and is a convenient shortcut for a loop of spaces.

For each solved pattern below, the program is written for $n = 5$ rows (change n to get a different size).

Pattern 1: Right-angled triangle of stars (for-for)

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print("*", end=" ")
    print()
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

Pattern 2: Inverted right triangle (while-while)

```
n = 5
i = n
while i >= 1:
    j = 1
    while j <= i:
        print("*", end=" ")
        j += 1
    print()
    i -= 1
```

Output

```
* * * * *
* * * *
* * *
* *
*
```

Pattern 3: Right-aligned triangle (spaces then stars)

```
n = 5
for i in range(1, n + 1):
    for s in range(n - i):          # n - i spaces
        print(" ", end=" ")
    for j in range(i):             # i stars
        print("*", end=" ")
    print()
```

Output

```
      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * *
* * * * *
```

Pattern 4: Pyramid — row i has $(n - i)$ spaces and $(2i - 1)$ stars.

```
n = 5
for i in range(1, n + 1):
    print(" " * (n - i), end="")
    for j in range(2 * i - 1):
        print("*", end="")
    print()
```

Output

```
      *
     ***
    *****
   *
  *
 *
*
```

Pattern 5: Diamond — a pyramid followed by an inverted pyramid.

```
n = 5
for i in range(1, n + 1):          # upper half
    print(" " * (n - i) + "*" * i)
for i in range(n - 1, 0, -1):     # lower half
    print(" " * (n - i) + "*" * i)
```

Output

```
      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * *
* * * * *
 * * * *
  * * *
   * *
    *
     *
      *
```

Pattern 6: Hollow square — print a star only on the border.

```
n = 5
for i in range(1, n + 1):
    for j in range(1, n + 1):
        if i == 1 or i == n or j == 1 or j == n:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
```

Output

```
* * * * *
*           *
*           *
*           *
*           *
* * * * *
```

Pattern 7: Number triangle — row i prints 1 to i.

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Pattern 8: Floyd's triangle — consecutive numbers using a running counter.

```
n = 5
num = 1
for i in range(1, n + 1):
    for j in range(i):
        print(f"{num:3}", end="")
        num += 1
    print()
```

Output

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Pattern 9: Number pyramid palindrome — 1, 121, 12321, ...

```
n = 5
for i in range(1, n + 1):
    print(" " * (n - i), end="")
    for j in range(1, i + 1):           # increasing part
        print(j, end=" ")
    for j in range(i - 1, 0, -1):       # decreasing part
        print(j, end=" ")
    print()
```

Output

```
    1
   1 2 1
  1 2 3 2 1
 1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
```

Pattern 10: Pascal's triangle — each value is $C(i, j)$, computed from the previous value in the row: $C(i, j) = C(i, j - 1) \times (i - j + 1) / j$.

```
n = 6
for i in range(n):
    print(" " * (n - i - 1), end="")
    c = 1
    for j in range(i + 1):
        print(f"{c:4}", end="")
        c = c * (i - j) // (j + 1)
    print()
```

Output

```
        1
       1  1
      1  2  1
     1  3  3  1
    1  4  6  4  1
   1  5 10 10  5  1
```

Pattern 11: Alphabet triangle (while-for)

```
n = 5
i = 1
while i <= n:
    for j in range(i):
        print(chr(65 + j), end=" ")
    print()
    i += 1
```

Output

```
A
A B
A B C
A B C D
A B C D E
```

Pattern 12: Alphabet pyramid — A, ABA, ABCBA, ... (for-while)

```
n = 5
for i in range(1, n + 1):
    print(" " * (n - i), end="")
    k = 0
    while k < i:                                # A B C ... up to the i-th letter
        print(chr(65 + k), end="")
        k += 1
    k = i - 2
    while k >= 0:                                # back down to A
        print(chr(65 + k), end="")
        k -= 1
    print()
```

Output

```
A
ABA
ABCBA
ABDCBA
ABCDEDCBA
```

Pattern 13: Large diamond with odd star counts — rows have 1, 3, 5, ..., 19 stars and then 17, 15, ..., 1 (n = 10). Each star is followed by a space, so every indentation step is two spaces.

```
n = 10
for i in range(1, n + 1):                        # upper half: rows 1..n
    print(" " * (n - i), end="")                # 2 spaces per missing position
    for j in range(2 * i - 1):                  # 2i - 1 stars
        print("*", end=" ")
    print()
for i in range(n - 1, 0, -1):                    # lower half: rows n-1..1
    print(" " * (n - i), end="")
    for j in range(2 * i - 1):
        print("*", end=" ")
    print()
```

[illegible]

Write a program to print each of the following patterns for $n = 5$ (your program should work for any n). Use a different loop combination (for-for, for-while, while-for, while-while) for at least four of them.

1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

```

1 2 3 4 5
1 2 3 4
1 2 3
1 2
1

```

```

5
5 4
5 4 3
5 4 3 2
5 4 3 2 1

```

Practice 4: Triangle of consecutive odd numbers

Expected output

```
1
3 5
7 9 11
13 15 17 19
21 23 25 27 29
```

Practice 5: Hollow pyramid

Expected output

```
  *
 * *
*   *
*     *
*****
```

Practice 6: Inverted pyramid of stars

Expected output

```
*****
*****
*****
***
*
```

Practice 7: Hourglass (sandglass)

Expected output

```
* * * * *
* * * *
* * *
* *
*
* *
* * *
* * * *
* * * * *
```

Practice 8: Butterfly

Expected output

```
*           *
**          **
***         ***
****        ****
*****       *****
****        ****
***         ***
**          **
*           *
```

Practice 9: Letter repeated in each row

Expected output

```
A
B B
C C C
D D D D
E E E E E
```

Practice 10: Reverse alphabet triangle

Expected output

```
E
E D
E D C
E D C B
E D C B A
```

Practice 11: Letter "X" (cross) using stars, n = 5

Expected output

```
*   *
*   *
  *
*   *
*   *
```

Practice 12: Number diamond (n = 4)

Expected output

```
    1
  1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
  1 2 3 2 1
    1 2 1
      1
```